



FACULTE DES SCIENCES
DEPARTEMENT D'INFORMATIQUE

Filière Sciences Mathématiques et Informatique (SMI)

MEMOIRE DE FIN D'ETUDES

Réalisé par :

JELLAOUI Bilal & BOUFFI Oussama

Encadré par :

Mme. LOUZAOUI Khadija

Intitulé

Classification Multi-Classes des images en IA avec Python

Soutenu le 23/07/2025 devant le Jury composé de :

Pr. LOUZAOUI Khadija : encadrante

Faculté des Sciences, Kénitra

Pr. BENLHACHMI Khalid : examinateur

Faculté des Sciences, Kénitra

Remerciements

Avant toute chose, nous tenons à exprimer notre plus profonde gratitude envers **Allah**, le Tout Puissant. C'est grâce à sa volonté et ses bénédictions que nous avons pu mener à bien la réalisation de ce pfe.

En second, Nous tenons à exprimer notre profonde gratitude à **Mme Khadija Louzaoui**, notre encadrante, pour son soutien exceptionnel tout au long de ce projet de fin d'études. Ses précieux conseils, sa disponibilité constante et son expertise ont été déterminants dans la réalisation de ce travail. Son accompagnement rigoureux et ses orientations éclairées nous ont permis de surmonter les défis techniques et méthodologiques rencontrés durant cette projet. Que Madame **Louzaoui** trouve ici l'expression de notre sincère reconnaissance pour son engagement et sa générosité intellectuelle.

Nous remercions cordialement l'examineur Benlhachmi Khalid pour l'intérêt qu'il a porté à notre travail en acceptant de l'évaluer.

Un immense merci à nos familles, pour leur amour inconditionnel, leurs bénédictions et leur soutien indéfectible, qui ont été une source inépuisable de réconfort et de soutien malgré la distance.

À nos amis(e), vous avez été des piliers solides, présents à chaque étape. Merci pour votre soutien sans faille, vos motivations, et vos encouragements.

Enfin, à toutes ces âmes bienveillantes croisées sur ce chemin, merci pour votre soutien inattendu qui a rendu ce voyage encore plus mémorable. Votre contribution a donné vie à ces pages.

Abstract

Artificial intelligence (AI) classification involves assigning predefined categories to data based on their features, using supervised learning algorithms. These algorithms enable systems to recognize patterns from labeled data. Once trained, the model can classify new inputs. In this study, convolutional neural networks (CNNs), specialized in visual pattern recognition, were selected to classify animal images. Architectural parameters, including the number of convolutional layers, pooling layers, fully connected neurons, and training epochs, were carefully optimized. Results highlighted the critical influence of these parameters and dataset size in determining the quality of the obtained classifications. despite challenges such as complex backgrounds.

Keywords: AI, CNN, Image Classification, Supervised Learning, Model Optimization.

Résumé

La classification en intelligence artificielle (IA) implique l'attribution de catégories prédéfinies à des données en fonction de leurs caractéristiques, à l'aide d'algorithmes d'apprentissage supervisé. Ces algorithmes permettent à un système d'identifier des motifs à partir de données étiquetées. Une fois entraîné, le modèle peut classer de nouvelles données. Dans cette étude, les réseaux de neurones convolutifs (CNN), spécialisés dans la reconnaissance de formes visuelles, ont été choisis pour classer des images d'animaux. Des paramètres architecturaux, tels que le nombre de couches de convolution, de pooling et de neurones entièrement connectés, ainsi que le nombre d'époques d'entraînement, ont été optimisés. Les résultats ont souligné l'impact critique de ces paramètres et de la taille du dataset sur la qualité des classifications obtenues malgré des défis comme la complexité des arrière-plans.

Liste des figures

Figure 1: Voisinage à 4.....	10
Figure 2: Voisinage à 8.....	10
Figure 3 : difference entre apprentissage automatique (machine learning) et deep learning..	14
Figure 4: schéma d'un RBF.....	17
Figure 5: Difference entre IA, Machine Learning, Deep Learning.....	26
Figure 6: Neurone artificiel.....	28
Figure 7: Différents types de fonctions d'activations.....	29
Figure 8: Perceptron simple.....	30
Figure 9: Perceptron multicouches (MLP).....	30
Figure 10: Exemple d'apprentissage profond.....	35
Figure 11: Architecture standard d'un réseau de neurone convolutionnel	36
Figure 12: Une couche du CNN en 3 dimensions. (Vert = volume d'entrée, bleu = volume du champ récepteur ,gris = couche de CNN, cercles = neurones artificiels indépendants).....	38
Figure 13 : Pooling avec un filtre 2x2 et un pas de 2	41
Figure 14: Fonction RELU	41
Figure 15: Modèle de réseau neuronal par Dropout [25]	44
Figure 16 : Exemple de Configuration	50
Figure 17: logo de python	52
Figure 18 : Configuration du modele	56
Figure 19: Code Source.....	57
Figure 20: Tableau dans une matrice de confusion.	59
Figure 21: Courbe ROC	60
Figure 22: Matrice de Confusion.....	60
Figure 23: Synthèse des métriques par classe.....	62
Figure 24: Précision et Erreur	62
Figure 25: Structure du dossier de l'application	66

Table des matières

<i>Remerciements.....</i>	<i>.....</i>
<i>Abstract.....</i>	<i>ii</i>
<i>Résumé</i>	<i>iii</i>
<i>Liste des figures.....</i>	<i>iv</i>
<i>Introduction Générale.....</i>	<i>1</i>
<i>Chapitre 1 : Un cadre général de projet.....</i>	<i>3</i>
1.1 Introduction	4
1.2 Motivations de la classification d'images.....	4
1.3 Notions fondamentales	5
1.3.1 Approches en intelligence artificielle	5
1.3.2 Concepts de base des images.....	8
1.4 Défis de la classification multiclasse.....	11
1.4.1 Définition de la classification multi-classes	11
1.4.2 Complexités de la classification multiclasse	12
1.4.3 Limites des méthodes traditionnelles de classification.....	13
1.5 Classification d'images.....	13
1.5.1 Apprentissage automatique(Machine Learning).....	13
1.5.2 Réseaux de neurones	14
1.6 Fondements des Réseaux de Neurones Convolutifs (CNN)	15
1.6.1 Convolution	15
1.6.2 Choix des CNN pour notre étude	16
1.6.3 Les différents types de réseaux de neurones	16
1.7 Domaines d'utilisation de l'intelligence artificielle	18
1.8 Conclusion	20
<i>Chapitre 2 : Intelligence Artificielle et Deep Learning.....</i>	<i>21</i>
2.1 Introduction	22
2.2 Fondements de l'intelligence artificielle.....	23
2.2.1 Évolution historique vers le Deep Learning	23
2.2.2 Amélioration de l'intelligence artificielle	24
2.2.3 La nécessité des réseaux de neurones en IA.....	25

2.3	Réseaux de neurones.....	27
2.3.1	Principes des neurones artificiels	27
2.3.2	Fonction d'activation	28
2.3.3	Types de perceptron	29
2.3.4	Mécanismes d'apprentissage.....	31
2.4	Apprentissage profond (Deep Learning)	33
2.4.1	Notions de base du Deep Learning.....	34
2.4.2	Comment fonctionne le deep Learning ?.....	35
2.4.3	Quelques algorithmes de deep Learning	35
2.5	Les réseaux de neurones convolutionnels (CNN)	36
2.5.1	Architecture de réseaux de neurone convolutionnel :.....	37
2.5.2	Exemple de modèles de CNN.....	42
2.6	Choix des paramètres.....	43
2.6.1	Nombre de filtres :.....	43
2.6.2	Forme du filtre :.....	43
2.6.3	Forme du Max Pooling :.....	43
2.7	Méthodes de régularisation.....	44
2.7.1	Empirique :	44
2.7.2	Explicite :.....	45
2.8	Conclusion	46
<i>Chapitre 3 : Classification d'images – Implémentation.....</i>		<i>47</i>
3.1	Introduction	48
3.2	Prétraitement des données.....	49
3.2.1	Normalisation des images.....	49
3.2.2	Augmentation des données (rotation, zoom, etc.)	49
3.3	Description du dataset.....	50
3.3.1	Source et structure du dataset	50
3.3.2	Répartition des classes.....	51
3.3.3	Exemple des images	52
3.4	Logiciels et librairies utilisés dans l'implémentation	52
3.4.1	Langage utilise.....	52
3.4.2	Bibliothèque Utilises	53
3.4.3	Configuration utilisé dans l'implémentation	54
3.5	Architecture du modèle	55
3.5.1	Choix du modèle (séquentiel vs graphe)	55
3.5.2	Configuration des couches (convolution, pooling, dense).....	55
3.5.3	Code Source et explication de model	57

3.6	Entraînement du modèle.....	57
3.6.1	Paramètres d'entraînement (learning rate, batch size).....	57
3.6.2	Surveillance des métriques (loss, accuracy).....	58
3.7	Évaluation des performances	58
3.7.1	Indicateurs de performance en classification.....	58
3.7.2	Matrice de confusion : Analyse des erreurs.....	60
3.7.3	Interprétation des faux positifs/négatifs	61
3.8	Résultats obtenus et discussion	62
3.8.1	Courbes d'apprentissage (train vs validation)	62
3.9	Conclusion	63
Chapitre 4 : Présentation de l'application mobile.....		64
4.1	Introduction	65
4.2	Pourquoi le Nom ZooBeco ?	65
4.3	Structure du dossier	65
4.4	Les outils utilisés	67
4.4.1	Kivy	67
4.4.2	Langage Kv	67
4.4.3	Exécution de l'application.....	68
4.5	Capture d'écran de l'interface	68
4.6	Pourquoi utiliser seuil de confiance	71
4.7	Conclusion	72
Conclusion générale		73
Perspectives		74
Bibliographie		75

Introduction Générale

L'**intelligence artificielle (IA)** et le **deep Learning** ont profondément transformé la vision par ordinateur, offrant des solutions innovantes pour des problèmes complexes tels que la classification d'images. Ce projet de fin d'études s'articule autour du développement d'un modèle de réseau de neurones convolutifs (CNN) capable de classer six espèces animales : Buffalo, chat, chien, éléphant, oiseau et zèbre. L'objectif principal est d'explorer les défis scientifiques et techniques liés à la distinction d'animaux morphologiquement proches, comme la différenciation entre un chat et un chien, ou la reconnaissance d'un chien dans des poses variées. Ce travail s'inscrit dans une démarche à la fois théorique, en étudiant les limites des architectures CNN, et pratique, en proposant des solutions reproductibles pour des applications réelles.

La classification multi-classe d'images animales présente des enjeux majeurs dans des domaines tels que l'écologie, la recherche en biodiversité ou l'éducation. Par exemple, un outil automatisé pourrait accélérer l'identification d'espèces menacées dans des réserves naturelles, soutenant ainsi les efforts de conservation. Selon l'Union internationale pour la conservation de la nature (UICN), près de 30 % des espèces évaluées sont menacées d'extinction, ce qui souligne l'urgence de développer des technologies d'analyse rapide et précise. Par ailleurs, ce projet pourrait servir de support pédagogique pour sensibiliser le public à la diversité animale, en illustrant concrètement les avancées de l'IA.

La problématique centrale réside dans la capacité du modèle à distinguer des espèces partageant des caractéristiques visuelles similaires, malgré des variations contextuelles telles que l'éclairage, les angles de prise de vue ou les arrière-plans complexes. Par exemple, un éléphant partiellement occulté par la végétation pourrait être confondu avec un Buffalo, tandis qu'un oiseau en vol, avec un arrière-plan flou, pourrait être mal classé en raison du manque de détails. Ces défis sont amplifiés par des contraintes techniques, comme le déséquilibre des jeux de données publics ou la nécessité d'optimiser les performances pour des applications temps réel.

Pour répondre à ces enjeux, **la méthodologie** adoptée combine une approche théorique rigoureuse et une implémentation pratique. Le modèle CNN est entraîné sur des datasets

variés (Kaggle, Image.cv, etc.), enrichis par des techniques d'augmentation de données (rotation, zoom, ajustement de luminosité) pour améliorer sa robustesse. L'architecture du réseau intègre des couches de convolution pour extraire des motifs locaux (rayures, textures), des mécanismes de régularisation (dropout) pour éviter le surapprentissage. Les performances sont évaluées via des métriques standard (précision, rappel, F1-score) et une analyse détaillée des erreurs à l'aide d'une matrice de confusion.

Ce mémoire est structuré en quatre chapitres, reflétant une progression logique depuis les bases **théoriques** jusqu'aux **applications concrètes** :

Le premier chapitre établit le cadre général, en présentant les concepts fondamentaux de l'IA, les défis de la classification multiclasse et les avantages des CNN.

Le deuxième chapitre approfondit les mécanismes du deep Learning, en explorant les architectures de réseaux de neurones et les méthodes d'optimisation.

Le troisième chapitre détaille l'implémentation pratique du modèle, incluant le prétraitement des données, l'entraînement et l'évaluation des résultats.

Enfin, **le quatrième chapitre** présente une application mobile opérationnelle, démontrant comment le modèle peut être intégré dans un outil accessible au grand public.

Ce projet illustre comment l'IA peut contribuer à des enjeux sociétaux, tout en soulignant l'importance d'une réflexion critique sur les limites technologiques et éthiques. Les résultats obtenus, bien que prometteurs, ouvrent la voie à des améliorations futures, comme l'intégration de modèles pré-entraînés ou l'exploration de données multimodale.

Chapitre 1 : Un cadre général de projet

- *Introduction*
- *Motivations de la classification des images...*
- *Notions fondamentales*
- *Défis de la classification multiclasse*
- *Classification des images*
- *Fondements des Réseaux de Neurones Convolutifs (CNN)*
- *Domaines d'utilisation de l'JA*
- *Conclusion*

1.1 Introduction

La classification automatique des images consiste à attribuer automatiquement une classe à une image à l'aide d'un système de classification. On retrouve ainsi la classification d'objets, de scènes, de textures, la reconnaissance de visages, d'empreintes digitale et de caractères. Dans ce projet, nous voulons apprendre à une machine à distinguer six animaux différents : le buffle, le chat, le chien, l'éléphant, l'oiseau et le zèbre. Pourquoi est-ce utile ? Par exemple, cela pourrait aider à identifier des espèces dans des réserves naturelles, à trier des photos personnelles, ou même à améliorer des systèmes de sécurité.

Dans ce chapitre, nous allons expliquer les bases nécessaires pour comprendre comment fonctionne cette technologie. Nous commencerons par les motivations de la classification d'images, puis nous découvrirons des notions simples sur l'intelligence artificielle (IA) et les caractéristiques des images (comme les pixels ou les couleurs). Nous parlerons aussi des défis quand on doit classer plusieurs animaux à la fois, et pourquoi les méthodes traditionnelles ne suffisent pas. Enfin, nous présenterons les réseaux de neurones, des outils puissants inspirés du cerveau humain, qui permettent de résoudre ces problèmes avec plus de précision.

1.2 Motivations de la classification d'images

La classification d'images a pour objectif de développer un système capable d'attribuer automatiquement une catégorie à une image. Ce type de système permet d'automatiser des tâches qui, réalisées manuellement par un humain, peuvent être longues, fatigantes et exigeantes, notamment lorsqu'il s'agit de traiter un grand volume de données visuelles.

Les domaines d'application de la classification automatique d'images sont nombreux. En agriculture, elle permet par exemple d'identifier des maladies sur les plantes ou d'estimer la maturité des récoltes. Dans le domaine du patrimoine culturel, elle aide à classer des objets anciens ou à analyser des œuvres historiques. Elle est également utilisée dans le domaine de la sécurité pour détecter des comportements suspects ou des objets dangereux, ainsi que dans l'industrie pour identifier des défauts de fabrication ou trier automatiquement des pièces sur les lignes de production. Même si la reconnaissance d'images semble naturelle pour l'être humain grâce à sa capacité à s'adapter à différentes conditions comme les angles de vue, l'éclairage ou les occultations partielles. La croissance rapide des bases de données rend l'analyse manuelle difficile, voire impossible. Avec

l'évolution des technologies d'acquisition (caméras, capteurs, microscopes) et de stockage, les images sont de plus en plus nombreuses et riches en information. Dans ce contexte, la charge de travail, la concentration nécessaire et la répétitivité des tâches deviennent des limites importantes pour une analyse humaine efficace.

À l'inverse, pour un ordinateur, une image est simplement un ensemble de valeurs numériques, ce qui complique son interprétation. Ainsi, concevoir des systèmes capables de comprendre et de classer automatiquement ces données visuelles constitue un défi majeur dans le domaine de l'intelligence artificielle.

Le point commun à toutes ces applications est qu'elles nécessitent la mise en place d'une chaîne de traitement à partir des images disponibles composée de plusieurs étapes afin de fournir en sortie une décision. Chaque étape de la mise en place d'un tel système de classification nécessite la recherche de méthodes appropriées pour une performance globale optimale à savoir la phase d'extraction de caractéristiques et la phase d'apprentissage. Typiquement, nous disposons de données images desquelles il nous faut extraire des informations pertinentes traduites sous formes de vecteurs numériques. Cette phase d'extraction nous permet de travailler dans un espace numérique. Il s'agit ensuite d'élaborer dans la phase d'apprentissage, à partir de ces données initiales, une fonction de décision pour décider de l'appartenance d'une donnée nouvelle à l'une des classes en présence [1].

1.3 Notions fondamentales

1.3.1 Approches en intelligence artificielle

1.3.1.1 Définition

L'intelligence artificielle est un ensemble de techniques permettant de résoudre des problèmes complexes en s'inspirant de mécanismes cognitifs humains, agissant de manière rationnelle en fonction de faits, données et expériences, et capables d'atteindre de manière optimale un ou plusieurs objectifs donnés [2].

Le but de l'intelligence artificielle est double. D'une part, l'IA s'attache à résoudre des problèmes qui relèvent d'activités humaines ou animales de nature variée : perception, planification, interprétation de données, diagnostic, prise de décision, compréhension du langage, conception. D'autre part, l'IA cherche à mieux comprendre et modéliser

l'intelligence [3]. Elle se rapproche ainsi des sciences cognitives dont elle s'inspire par ailleurs pour la conception de modèles (mémoire, raisonnement, apprentissage).

L'intelligence artificielle est un mélange de mathématiques et d'informatique. Ce n'est pas de la magie, mais bien des calculs. Prenons l'exemple d'une voiture autonome, une intelligence artificielle est capable de conduire une voiture toute seule, à condition qu'un ingénieur programme les commandes utiles pour passer les vitesses, freiner, reculer et les règles du code de la route [2].

1.3.1.2 Rôle de l'IA dans la reconnaissance d'animaux

Les scientifiques peuvent utiliser l'IA de différentes manières pour évaluer l'état de santé des différentes espèces, déterminer quels types d'animaux sauvages sont en difficulté et planifier les moyens de les aider.

Reconnaissance des espèces : L'IA peut reconnaître différents types d'animaux. Les chercheurs peuvent installer des caméras à différents endroits ou utiliser des caméras sur des drones pour prendre des photos d'une zone, puis demander à l'IA de regarder toutes les photos et de leur dire quels types d'animaux vivent à cet endroit et combien il y en a pour chacun d'entre eux. L'IA peut même reconnaître le visage de certains animaux, tout comme vous reconnaissez un ami lorsque vous le voyez à l'école.

Suivi des animaux : Parfois, les scientifiques attrapent des animaux et les dotent de dispositifs de suivi afin de savoir où ils vont. L'IA peut examiner toutes les données fournies par ces dispositifs et déterminer où les animaux aiment vivre, où ils ont tendance à migrer et quand ils se déplacent d'un endroit à l'autre. Elle peut alors prédire où et quand ils se rendront ensuite.

Protection des habitats : Lorsque nous utilisons l'IA pour déterminer où les animaux vivent et où ils peuvent se déplacer, nous savons quelles zones nous devons protéger et dont nous devons éloigner les gens. L'homme peut également utiliser l'IA pour surveiller une zone afin de détecter les signes de problèmes : Nous pouvons envoyer des drones au-dessus d'une zone pour prendre des photos, puis demander à l'IA de nous dire ce qui a changé dans la zone et pourquoi. Par exemple, cette technologie peut repérer l'abattage d'arbres ou le tracé de chemins dans une zone où les gens ne sont pas censés faire ce genre de choses. L'IA peut également prédire où les animaux risquent d'avoir des problèmes. Par exemple, si un troupeau d'éléphants s'approche d'un village, l'IA peut envoyer un

avertissement aux habitants. Cela permet d'éviter les rencontres dangereuses entre les animaux et les humains, et de garantir la sécurité des uns et des autres. [4]

1.3.1.3 Enjeux et contraintes de l'intelligence artificielle

L'Intelligence Artificielle (IA) connaît un essor fulgurant et transforme de nombreux secteurs. Bien que des avancées majeures aient été accomplies, plusieurs défis persistent pour exploiter pleinement son potentiel.

Apprentissage et Entraînement des Modèles : L'efficacité des algorithmes d'IA dépend largement de la qualité et de la quantité des données disponibles. Or, l'accès à des jeux de données diversifiés et non biaisés reste un obstacle majeur. Par ailleurs, il est crucial d'assurer la représentativité des données pour éviter des modèles discriminants ou peu performants.

Interprétabilité des Décisions : La complexité des modèles d'IA, notamment ceux basés sur l'apprentissage profond, rend souvent leurs décisions opaques. Développer des méthodes d'explication transparentes est essentiel pour renforcer la confiance et permettre une adoption responsable de ces technologies.

L'intelligence artificielle prend une place de plus en plus importante dans de nombreux domaines de la société, ce qui soulève plusieurs questions éthiques et juridiques. L'un des principaux enjeux concerne la protection des données personnelles. Les systèmes d'IA utilisent souvent de grandes quantités de données, ce qui peut mettre en danger la vie privée des individus si ces informations ne sont pas bien sécurisées. Un autre point important est le manque de transparence des algorithmes. Les décisions prises par l'IA peuvent avoir des conséquences importantes sur les personnes, mais elles sont souvent difficiles à comprendre. Il est donc nécessaire que les systèmes d'IA soient capables d'expliquer clairement leurs décisions, afin de maintenir la confiance des utilisateurs et d'éviter les erreurs ou les injustices. L'équité dans les décisions automatiques est aussi un enjeu central. Les algorithmes peuvent, sans le vouloir, reproduire ou renforcer les biais présents dans les données utilisées pour les entraîner. Cela peut conduire à des discriminations. Pour éviter cela, il est important de mettre en place des mécanismes permettant de détecter et de corriger ces biais, afin que les décisions prises soient justes pour tous. En conclusion, le développement de l'intelligence artificielle pose des défis majeurs sur les plans éthique et juridique. Il est essentiel d'adopter des règles claires et de

suivre des principes responsables pour garantir une utilisation de l'IA respectueuse des droits fondamentaux des individus.

1.3.2 Concepts de base des images

1.3.2.1 Définition d'une image

Une image est la projection d'une scène réelle ou virtuelle sur une surface plane, réalisée par la convergence de rayons lumineux qui ont interagi avec les objets présents dans l'espace. Elle peut être obtenue à l'aide de dispositifs optiques, tels que des lentilles, ou par des moyens numériques, tels que les capteurs d'image.

Une image peut être définie comme une fonction bidimensionnelle $f(x, y)$, où x et y sont les coordonnées spatiales, et l'amplitude de f à chaque paire de coordonnées (x, y) est appelée l'intensité ou le niveau de gris de l'image à ce point [5].

La fonction Image peut se représenter sous la forme suivante :

$$\begin{aligned} IF : \mathbb{R}^2 &\rightarrow \mathbb{R} && \text{Avec } \mathbb{R} : \text{ensemble des réelles.} \\ (x, y) &\rightarrow IF(x, y) && x \text{ et } y : \text{Deux variables réelles.} \end{aligned}$$

1.3.2.2 Les différents types du format d'image

Une image RVB (Rouge, Vert, Bleu) est représentée par trois matrices de dimensions identiques, chacune associée à l'intensité d'une couleur particulière. Ces intensités suivent le même format que pour les images en niveaux de gris, mais appliquées individuellement à chaque couleur. Chaque matrice correspond ainsi à l'intensité du rouge, du vert ou du bleu. Avec 256 niveaux d'intensité possibles pour chacune de ces couleurs, cela permet de générer 256^3 couleurs différentes pour chaque pixel, en fonction des valeurs choisies pour chaque composant de couleur.

- Image d'intensités : C'est une matrice dans laquelle chaque élément est un réel compris entre 0 (noir) et 1 (blanc). On parle aussi d'image en niveaux de gris, car les valeurs comprises entre 0 et 1 représentent les différents niveaux de gris.
- Image binaire : Une image binaire est une matrice rectangulaire dans l'élément valent 0 ou 1. Lorsque l'on visualise une telle image, les 0 sont représentés par du noir et les 1 par du blanc.

1.3.2.3 Caractéristiques de l'image

L'image est un ensemble structuré d'informations caractérisé par les paramètres suivants :

Pixel : Le **pixel** est l'unité de base d'une image numérique, représentant un point (x, y) sur le plan de l'image. Il véhicule une information visuelle, comme une intensité de gris ou une couleur. Dans une image monochrome, il est codé sur un octet, tandis que dans une image couleur (RVB), il est représenté par trois octets, un pour chaque composante rouge, verte et bleue. La quantité d'information par pixel détermine la qualité visuelle de l'image et influe sur la mémoire nécessaire à son traitement.

Dimension et Résolution : C'est la taille de l'image. Cette dernière se présente sous forme de matrice dont les éléments sont des valeurs numériques représentatives des intensités lumineuses (pixels). Le nombre de lignes de cette matrice multiplié par le nombre de colonnes nous donne le nombre total de pixels dans une image [6]. Par contre, la résolution est définie par un nombre de pixels par unité de longueur de l'image à numériser en dpi (dots per inch) ou ppp (points par pouce). On parle de définition pour un écran et de résolution pour une image. La résolution d'une image correspond au niveau de détail qui va être représenté sur cette image. Pour la numérisation il faut considérer les 2 équations suivantes :

- $(X \times \text{résolution}) = x \text{ pixels}$
- $(Y \times \text{résolution}) = y \text{ pixels}$

Où :

- X et Y représentent la taille (pouce ou cm, un pouce=2,54 centimètres) de la structure à numériser.
- Résolution représente la résolution de numérisation, x et y représentent la taille (en pixels) de l'image [7].

Voisinage : Le plan de l'image est divisé en termes de formes rectangulaires ou hexagonales permettant ainsi l'exploitation de la notion de voisinage (voir figure 1). Le voisinage d'un pixel est formé par l'ensemble des pixels qui se situent autour de ce même pixel. On définit aussi l'assiette comme étant l'ensemble de pixels définissant le voisinage pris en compte autour d'un pixel. On distingue deux types de voisinage :

Voisinage à 4 : On ne prend en considération que les pixels qui ont un coté commun avec le pixel considéré.

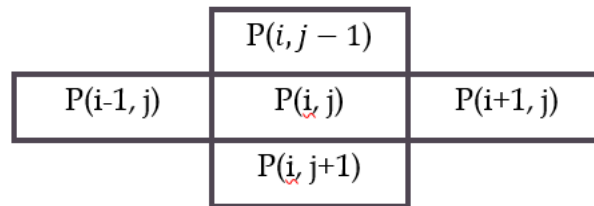


Figure 1: Voisinage à 4

Voisinage à 8 : On prend en compte tous les pixels qui ont au moins un point en liaison avec le pixel considéré

$P(i-1, j-1)$	$P(i, j-1)$	$P(i+1, j-1)$
$P(i-1, j)$	$P(i, j)$	$P(i+1, j)$
$P(i-1, j+1)$	$P(i, j+1)$	$P(i+1, j+1)$

Figure 2: Voisinage à 8

Niveau de gris : C'est la valeur d'intensité lumineuse d'un pixel. Cette valeur peut aller du noir (0) jusqu'au blanc (255) en passant par les nuances qui sont contenues dans l'intervalle $[0, 255]$. Elle correspond en fait à la quantité de la lumière réfléchie. Pour 8 bits, on dispose de 256 niveaux de gris dont 40 sont reconnus à l'œil nue. Plus le nombre de bit est grand plus les niveaux sont nombreux et plus la représentation est fidèle.

Contraste : Le **contraste** désigne la différence d'intensité lumineuse entre deux zones d'une image, généralement entre les régions claires et sombres. Il est déterminé en fonction des **luminances** respectives de ces zones, et permet de mettre en évidence les variations visuelles, améliorant ainsi la perception des détails.

Si $L1$ et $L2$ sont les degrés de luminosité respectivement de deux zones voisines $A1$ et $A2$ d'une image, le contraste C est défini par le rapport : [8]

$$C = \frac{L1 - L2}{L1 + L2}$$

Luminance : C'est le degré de luminosité des points de l'image. Elle est définie aussi comme étant le quotient de l'intensité lumineuse d'une surface par l'aire apparente de cette surface, pour un observateur lointain, le mot luminance est substitué au mot brillance, qui correspond à l'éclat d'un objet. Une bonne luminance se caractérise par :

- Des images lumineuses (brillantes) ;
- Un bon contraste : il faut éviter les images où la gamme de contraste tend vers le blanc ou le noir; ces images entraînent des pertes de détails dans les zones sombres ou lumineuses.
- L'absence de parasites.

Bruit : Un bruit (parasite) dans une image est considéré comme un phénomène de brusque variation de l'intensité d'un pixel par rapport à ses voisins, il provient de l'éclairage des dispositifs optiques et électroniques du capteur [9].

Contours et textures : Les contours représentent la frontière entre les objets de l'image, ou la limite entre deux pixels dont les niveaux de gris représentent une différence significative. Les textures décrivent la structure de ceux-ci. L'extraction de contour consiste à identifier dans l'image les points qui séparent deux textures différentes [7].

1.4 Défis de la classification multiclasse

L'objectif de la classification d'images est d'élaborer un système capable d'affecter automatiquement une image à une classe. Ainsi, ce système permet d'effectuer une tâche d'expertise qui peut s'avérer coûteuse à acquérir pour un être humain en raison notamment de contraintes physiques comme la concentration, la fatigue ou le temps nécessaire à un volume important d'images.

1.4.1 Définition de la classification multi-classes

La classification multi-classe est une approche d'apprentissage supervisé qui vise à associer chaque donnée d'entrée à une seule classe parmi plusieurs catégories possibles (généralement trois ou plus). Le processus commence par l'apprentissage à partir d'un ensemble d'exemples annotés, où chaque instance est liée à une étiquette. Ces étiquettes, souvent textuelles au départ, sont converties en valeurs numériques à l'aide de techniques de transformation comme *TermTransform*, afin d'être utilisables par les algorithmes de classification. Le modèle ainsi entraîné devient un classifieur, capable de prédire la classe

appropriée pour de nouvelles données non étiquetées. Dans ce type de classification, les catégories sont exclusives : une même instance ne peut appartenir qu'à une seule classe à la fois. Cette méthode est couramment appliquée dans des situations où un choix unique doit être fait entre plusieurs options.

Parmi les cas d'usage typiques, on peut citer :

- La catégorisation des vols selon leur ponctualité : « en avance », « à l'heure » ou « en retard » ;
- L'analyse des critiques de films pour déterminer si elles sont « positives », « neutres » ou « négatives » ;
- La classification des avis d'hôtel selon des critères comme « emplacement », « prix » ou « propreté ».

Dans un autre exemple, un système de reconnaissance d'images peut être entraîné à distinguer entre différents objets comme des avions, des voitures ou des bateaux. Si une image représente un avion, le modèle devra la classer correctement sans la confondre avec d'autres catégories.

1.4.2 Complexités de la classification multiclasse

La classification multi-classes soulève des défis complexes, notamment liés à la distribution déséquilibrée des données. Lorsque certaines classes dominent numériquement (exemple : 10 000 images de chiens vs 100 images de pangolins), les modèles apprennent involontairement à privilégier les classes majoritaires, minimisant l'impact des exemples rares sur la fonction de coût. Ce biais se traduit par une chute drastique du rappel pour les classes minoritaires, compromettant l'utilité pratique du modèle dans des contextes critiques comme le diagnostic médical ou la surveillance écologique.

Par ailleurs, la complexité géométrique des frontières de décision s'accroît exponentiellement avec le nombre de classes. Dans un espace de features multidimensionnel, séparer des classes morphologiquement proches (exemple : espèces de papillons tropicaux) exige des modèles non linéaires et profondément structurés (exemple : réseaux de neurones convolutionnels), au risque de surapprentissage. Ce phénomène est exacerbé par la malédiction de la dimensionalité : en haute dimension, les données deviennent sparse, rendant les similarités inter-classes moins discernables.

1.4.3 Limites des méthodes traditionnelles de classification

Les méthodes traditionnelles de classification d'images, comme les SVM, les forêts aléatoires ou les k-plus proches voisins (k-NN), rencontrent des limites majeures dans des tâches complexes telles que la distinction entre espèces animales (Buffalo, chat, oiseau, etc.). Ces approches dépendent d'une extraction manuelle de caractéristiques (contours, textures), un processus rigide qui peine à capturer des détails contextuels ou hiérarchiques, comme différencier un chien d'un loup malgré leurs similitudes. De plus, la haute dimensionnalité des images (des milliers de pixels) entraîne une complexité calculatoire excessive, obligeant à réduire artificiellement les données et risquant de perdre des informations cruciales. Ces méthodes sont aussi vulnérables aux transformations géométriques (rotation, occlusion) et aux variations réalistes (éclairage, arrière-plans), limitant leur capacité à généraliser. Enfin, pour des problèmes multi-classes, leur précision chute face à des similarités inter-espèces (exemple : chat vs. Lynx) ou à un grand nombre de catégories. En Résumé, les méthodes classiques sont utiles pour des tâches simples, mais les CNN sont bien plus adaptés pour classer des animaux dans des images réelles, grâce à leur flexibilité et leur intelligence automatique.

1.5 Classification d'images

1.5.1 Apprentissage automatique(Machine Learning)

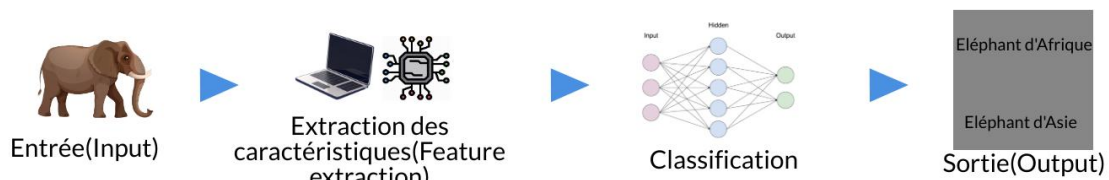
L'apprentissage automatique (le Machine Learning) est une technique de modélisation qui implique des données. Cette la définition peut être trop courte pour les débutants pour saisir ce que cela signifie. Alors laisse-moi élaborer un peu sur cela. L'apprentissage automatique est une technique qui permet de comprendre le « modèle » à partir des « données ». Ici, les données signifient littéralement des informations telles que documents, audio, images, etc. Le «modèle» est le produit final de l'apprentissage automatique [10] .

Dans les premiers temps des applications dites « intelligentes », de nombreux systèmes utilisaient des règles figées dans le code, basées sur des décisions de type « si... sinon... » pour traiter les données ou encore ajuster les entrées de l'utilisateur. Soit par exemple un filtre anti spam, son travail consiste à déplacer des messages électroniques entrants dans un dossier spécifique. Vous pourriez créer une liste noire de termes qui ferait en sorte qu'un message soit marqué comme étant un spam. Il s'agirait d'un exemple typique de système de règles « expert » servant à créer une application « intelligente ». De telles règles

décisionnelles, ciselées à la main, sont tout à fait applicables dans le cas de certaines applications, en particulier celles pour lesquelles les êtres humains ont une bonne compréhension des processus à modéliser. Cependant, cette façon de procéder présente deux inconvénients majeurs : [11]

- La logique nécessaire à la prise de décision est spécifique à un certain domaine et une certaine tâche. Modifier celle-ci, même légèrement, peut impliquer la réécriture de l'ensemble du système.
- Concevoir des règles nécessite une compréhension approfondie de la manière dont une décision devrait être prise par un expert humain.

Machine Learning



Deep Learning

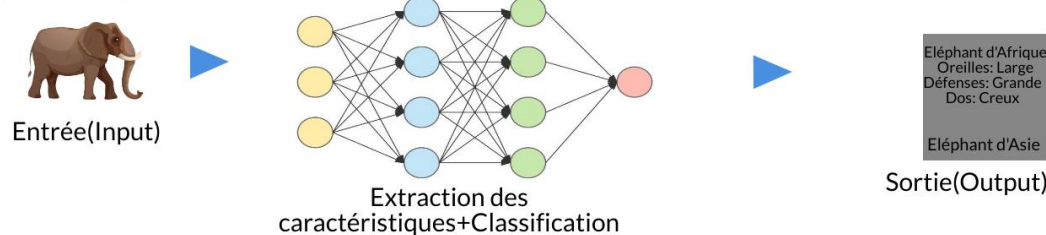


Figure 3 :différence entre apprentissage automatique (machine learning) et deep learning

1.5.2 Réseaux de neurones

Le problème de l'approche classique de la reconnaissance des images est qu'un bon extracteur de caractéristiques est très difficile à construire, et qu'il doit être repensé pour chaque nouvelle application. C'est là qu'intervient l'apprentissage profond ou deep Learning en anglais. C'est une classe de méthodes dont les principes sont connus depuis la fin des années 1980, mais dont l'utilisation ne s'est vraiment généralisée que depuis 2012, environ. L'idée est très simple : le système entraînable est constitué d'une série de modules, chacun représentant une étape de traitement. Chaque module est entraînable, comportant des paramètres ajustables similaires aux poids des classifieurs linéaires. Le système est entraîné de bout en bout : à chaque exemple, tous les paramètres de tous les modules sont ajustés de manière à rapprocher la sortie produite par le système de la sortie désirée. Le qualificatif profond vient de l'arrangement de ces modules en couches

successives. Pour pouvoir entraîner le système de cette manière, il faut savoir dans quelle direction et de combien ajuster chaque paramètre de chaque module. Pour cela il faut calculer un gradient, c'est-à-dire pour chaque paramètre ajustable, la quantité par laquelle l'erreur en sortie augmentera ou diminuera lorsqu'on modifiera le paramètre d'une quantité donnée. Le calcul de ce gradient se fait par la méthode de rétropropagation, pratiquée depuis le milieu des années 1980. Dans sa réalisation la plus commune, une architecture profonde peut être vue comme un réseau multicouche d'éléments simples, similaires aux classifieurs linéaires, interconnectés par des poids entraînaibles. C'est ce qu'on appelle un réseau neuronal multicouche.

Pourquoi neuronal ? Un modèle extrêmement simplifié des neurones du cerveau les voit comme calculant une somme pondérée et activant leur sortie lorsque celle-ci dépasse un seuil. L'apprentissage modifie les efficacités des synapses, les poids des connexions entre neurones. Un réseau neuronal n'est pas un modèle précis des circuits du cerveau, mais est plutôt vu comme un modèle conceptuel ou fonctionnel. Le réseau neuronal est inspiré du cerveau un peu comme l'avion est inspiré de l'oiseau.

Ce qui fait l'avantage des architectures profondes, c'est leur capacité d'apprendre à représenter le monde de manière hiérarchique. Comme toutes les couches sont entraînaibles, nul besoin de construire un extracteur de caractéristiques à la main. L'entraînement s'en chargera. De plus, les premières couches extrairont des caractéristiques simples (présence de contours) que les couches suivantes combineront pour former des concepts de plus en plus complexes et abstraits: assemblages de contours en motifs, de motifs en parties d'objets, de parties d'objets en objets, etc [12].

1.6 Fondements des Réseaux de Neurones Convolutifs (CNN)

1.6.1 Convolution

La fonction de convolution peut se résumer en deux points. Le premier est la détection de motifs (patterns) invariants dans les images. En d'autres termes, afin de partager équitablement les poids entre des caractéristiques similaires, la convolution doit pouvoir détecter les similitudes de formes indépendamment de leur emplacement ou de leur variation. La deuxième caractéristique de la convolution est l'interaction locale (« sparse interaction »). Le principe réside dans la capacité du réseau à détecter des objets dans des régions locales indépendamment de la globalité de l'image. En d'autres termes, chaque

valeur de sortie d'une couche dépend d'un petit nombre d'entrées, plutôt que de tout l'ensemble. En bref, la convolution tente de capturer l'essentiel de chaque entrée en regroupant les parties possibles séparément de l'image entière.

1.6.2 Choix des CNN pour notre étude

Les réseaux de neurones convolutifs (CNN) sont privilégiés pour la classification d'images, comme la reconnaissance d'animaux, car ils surmontent les limites des méthodes traditionnelles. Contrairement aux approches classiques (SVM, forêts aléatoires), qui reposent sur une extraction manuelle de caractéristiques (contours, textures), les CNN **apprennent automatiquement** les détails pertinents directement depuis les pixels, sans intervention humaine. Par exemple, un CNN identifie seul les rayures d'un zèbre ou les oreilles pointues d'un chat, même si l'image est floue, tournée, ou partiellement occultée. Cette capacité vient de leur architecture unique : les **couches de convolution** détectent des motifs locaux (bords, textures) en tout point de l'image, tandis que le **pooling** les rend insensibles aux déplacements ou aux changements d'échelle. De plus, les CNN gèrent efficacement la **haute dimensionnalité** des images (milliers de pixels) grâce au partage de paramètres entre filtres, évitant l'explosion computationnelle. Ils excellent aussi pour les problèmes complexes (distinguer un chien d'un loup) en capturant des caractéristiques hiérarchiques et abstraites via des couches profondes. Enfin, leur compatibilité avec **l'apprentissage par transfert** permet de réutiliser des modèles pré-entraînés (ex : ResNet) et de les adapter rapidement à de nouvelles tâches, même avec peu de données. Robustes aux variations réalistes (lumière, arrière-plan) et scalables pour des centaines de classes, les CNN dominent aujourd'hui dans des applications concrètes, comme les assistants visuels ou la surveillance écologique, offrant précision, flexibilité et efficacité.

1.6.3 Les différents types de réseaux de neurones

1.6.3.1 ADALINE :

L'Adaptive Linear Élément a la même architecture que le perceptron, mais en revanche, il possède des valeurs d'activations continues, une fonction d'activation linéaire est une règle d'apprentissage un peu développée que celle du perceptron, elle s'appelle la règle de WIDROW & HOFF, en effet cette loi minimise l'erreur quadrique, en utilisant la méthode du gradient. Contrairement à la règle du perceptron cette dernière opère sur les

exemples bien classés et mal classés en optimisant toujours les coefficients des hyperplans séparateurs.

Formellement, on définit la dynamique ainsi que la règle d'apprentissage comme suit :

1.6.3.2 Les réseaux de RBF (Radial Basis Fonction) :

Le réseau RBF est un réseau de neurones supervisé. Il s'agit d'une spécialisation d'un PMC. Un RBF est constitué uniquement de 3 couches (voir Figure 4)

- La couche d'entrée : elle retransmet les entrées sans distorsion.
- La couche RBF : couche cachée qui contient les neurones RBF.
- La couche de sortie : simple couche qui contient une fonction linéaire.

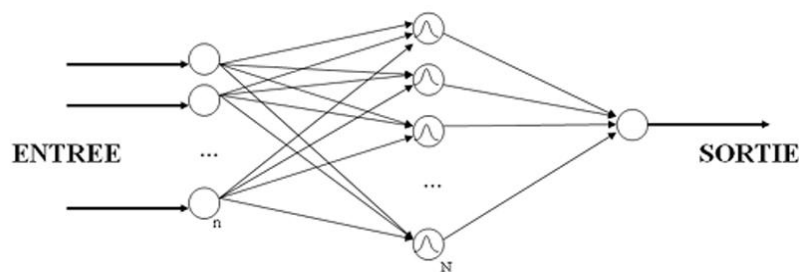


Figure 4: schéma d'un RBF

Chaque neurone RBF contient une gaussienne qui est centrée sur un point de l'espace d'entrée. Pour une entrée donnée, la sortie du neurone RBF est la hauteur de la gaussienne en ce point. La fonction gaussienne permet aux neurones de ne répondre qu'à une petite région de l'espace d'entrée, région sur laquelle la gaussienne est centrée. Donc il y a quatre paramètres principaux à régler dans un réseau RBF.

- Le nombre de neurones RBF (nombre de neurones dans l'unique couche cachée).
- La position des centres des gaussiennes de chacun des neurones.
- La largeur de ces gaussiennes.
- Le poids des connexions entre les neurones RBF et le(s) neurone(s) de sortie.

Toute modification d'un de ces paramètres entraîne directement un changement du comportement du réseau.

1.1.1.1 Réseaux de Hopfield :

Il s'agit d'un réseau constitué de neurones à deux états (-1 et 1, ou 0 et 1), dont la loi d'apprentissage est la règle de Hebb (1949), qui veut qu'une synapse améliore son activité si et seulement si l'activité de ses deux neurones est corrélée (c'est-à-dire que le poids d'une connexion entre deux neurones augmente quand les deux neurones sont activés au même temps).

1.1.1.1 Réseaux de Kohonen :

Contrairement aux réseaux de Hopfield où les neurones sont modélisés de la façon la plus simple possible, on recherche ici un modèle de neurone plus proche de la réalité. Ces réseaux sont inspirés des observations biologiques du fonctionnement des systèmes nerveux de perception des mammifères. Une loi de Hebb modifiée (tenant compte de l'oubli) est utilisée pour l'apprentissage. La connexion est renforcée dans le cas où les neurones reliés ont une activité simultanée et diminuée dans le cas contraire (alors qu'il ne se passait précédemment rien dans ce cas). Tous ces réseaux ont des applications dans la classification, le traitement d'image, l'aide à la décision et l'optimisation.

1.7 Domaines d'utilisation de l'intelligence artificielle

L'intelligence artificielle est présente en permanence dans notre vie quotidienne. D'un appareil connecté qui diffuse une playlist adaptée aux précédentes écoutes jusqu'à nos achats, on constate une avancée de la technologie à des niveaux multiples. Les domaines d'application ne cessent en effet de se diversifier, depuis le médical jusqu'aux transports et la sécurité, en passant par le commerce et les finances., elle a un impact considérable sur de nombreux domaines, dont certains sont mentionnés ci-dessous :

- **Les jeux :** L'intelligence artificielle a rapidement trouvé sa place dans l'univers des jeux vidéo. En effet, chaque jeu repose sur des programmes qui simulent des réactions intelligentes en réponse aux actions qui s'y produisent. Ainsi, la machine doit être capable d'envisager plusieurs scénarios possibles, en s'appuyant sur des connaissances heuristiques.

- **Programmation Automatique :** Il peut s'agir d'écrire des programmes qui écrivent des programmes résolvant des problèmes décrits en langage mathématique ou en langage naturel par exemple. (Ces programmes sont souvent moins élégants que

les nôtres mais peuvent être meilleurs !) Cette activité a été pendant un temps concurrencée par les systèmes experts. Elle en est maintenant complémentaire, de nombreuses bases de connaissances étant compilées, c'est-à-dire remplacées par des connaissances procédurales (programmes C par exemple) dans un souci d'efficacité [13].

-les systèmes experts : sont des programmes informatiques capables de reproduire les compétences d'un experts humains dans des domaines spécifiques. Ils assistent les utilisateurs dans la prise de décision en fournissant des conseils basés sur une base de connaissances spécialisées. En combinant matériel, programmation et données spécifiques, ils permettent d'obtenir des résultats fiables offrent des recommandations claires aux utilisateurs.

-Les robots Intelligents : La robotique n'est pas forcément de l'IA. Elle peut en être par l'aspect Reconnaissance des formes et Résolution de problèmes. Il peut être nécessaire de faire face à des situations imprévues.

Quelques applications sont particulièrement utiles : travaux dangereux, réparations dans les centrales nucléaires ou dans l'espace [13].

- Autres :

- o Interaction homme/machine
- o Médecine, Modélisation cognitive
- o Réseaux neuronaux et approche connexionnistes
- o Reconnaissance des formes et perception
- o Reconnaissance de la parole et des visages
- o Reconnaissance de l'écriture manuscrite... [13]

1.8 Conclusion

Dans ce premier chapitre, nous avons posé les bases pour comprendre la classification multiclasse d'images appliquée à six animaux. Nous avons vu que cette technologie repose sur des concepts clés comme l'intelligence artificielle, les caractéristiques des images (pixels, résolution), et les limites des méthodes classiques. Les réseaux de neurones convolutifs (CNN) se sont imposés comme une solution efficace pour analyser des détails complexes, comme les rayures d'un zèbre ou les plumes d'un oiseau, grâce à leur capacité à extraire des motifs locaux.

Ces connaissances préparent le terrain pour le chapitre suivant, qui approfondira l'IA et le deep learning. Nous explorerons comment les réseaux de neurones fonctionnent en détail : leur histoire, leur structure (comme les perceptrons multicouches), les fonctions d'activation qui les rendent intelligents, et les choix techniques pour optimiser leur performance (comme le nombre de filtres ou les méthodes de régularisation). Ce sera l'occasion de découvrir des modèles concrets de CNN et de comprendre leurs avantages et inconvénients dans la pratique.

Chapitre 2 : Intelligence Artificielle et Deep Learning

- *Introduction*
- *Fondements de l'intelligence artificielle*
- *Réseaux de neurones*
- *Deep Learning et Réseaux Convolutifs (CNN)*
- *Exemple de modèles de CNN*
- *Choix des Paramètres dans les Réseaux Convolutifs*
- *Méthodes de régularisation*
- *Conclusion*

2.1 Introduction

L'apprentissage profond ou « Deep Learning » est une branche de l'intelligence artificielle dérivée de l'apprentissage automatique ou « machine Learning ». Contrairement à la programmation qui consiste à exécuter des règles préétablies, ce type d'apprentissage permet aux machines d'apprendre de manière autonome. En fait, les machines sont capables de s'entraîner sur des données, de reconnaître des modèles et de prendre des décisions basées sur ces données sans intervention humaine.

L'apprentissage profond a été utilisé dans diverses applications, notamment la reconnaissance d'images et de la parole, le traitement du langage naturel et la conduite autonome. Il a permis d'atteindre des performances de pointe dans de nombreuses tâches et a été à l'origine de nombreuses avancées récentes dans le domaine de l'intelligence artificielle. Cependant, l'apprentissage profond nécessite également de grandes quantités de données et de ressources informatiques, ce qui en fait une technologie gourmande en ressources.

L'élément de base de l'apprentissage profond est le réseau neuronal qui se compose de neurones artificiels, ce dernier prend des valeurs d'entrée, applique des poids à ces entrées et produit une sortie. Dans un réseau neuronal profond, plusieurs couches de neurones sont empilées les unes sur les autres, chaque couche transformant l'entrée de la couche précédente jusqu'à ce qu'une sortie finale soit produite.

Ce chapitre explore les fondements de l'IA et les rouages du Deep Learning et son rôle central dans l'IA moderne. Nous commencerons par retracer l'évolution historique qui a conduit au Deep Learning, en analysant les différents types d'IA et leur portée. Nous expliquerons ensuite pourquoi les réseaux de neurones sont devenus incontournables pour traiter des données complexes, avant de plonger dans leur architecture, leurs mécanismes d'apprentissage et leurs composants clés (perceptrons, fonctions d'activation). Enfin, nous aborderons les réseaux convolutifs (CNN), leur conception, leurs paramètres optimaux et les méthodes pour éviter le surajustement. En comprenant ces mécanismes, vous saisirez comment le Deep Learning permet à l'IA de dépasser les limites des approches traditionnelles et de s'adapter à des défis toujours plus ambitieux.

2.2 Fondements de l'intelligence artificielle

2.2.1 Évolution historique vers le Deep Learning

L'intelligence artificielle (IA) n'est pas une simple invention, c'est une véritable odyssée humaine. Son histoire, ponctuée de percées fulgurantes et de moments de doute, reflète notre quête inlassable de reproduire l'intelligence humaine dans les machines.

Naissance officielle : Le terme "intelligence artificielle" a été utilisé pour la première fois en 1956 lors de la conférence de Dartmouth, organisée par John McCarthy, qui est considéré comme l'un des pionniers du domaine. Cette conférence a établi l'IA comme une discipline de recherche à part entière.

Le test de *TURING* : En 1950, c'est au tour des scientifiques d'imaginer des machines pensantes. Le mathématicien britannique, Alan Turing, père de l'informatique moderne, publie un article intitulé « l'ordinateur et l'intelligence », dans lequel il décrit comment savoir si une machine s'approche d'une intelligence humaine. Il appelle cette méthode « le jeu de l'imitation », plus connu aujourd'hui sous le nom de test de Turing. Depuis, les recherches sur l'IA ont connu successivement des périodes d'essor et de gel [14].

La méthode de résolution 1965 : La Méthode de Résolution (Robinson), cette méthode a donné naissance au langage PROLOG, ainsi que l'apparition des premiers chatbots comme c'est le cas du programme ELIZA créé en 1966. Ces chatbots tenaient le coup pendant des conversations avec quelques échanges mais ne passaient pas le test de Turing [2].

Les années 1970 : La fin des années 60 marque une période de crise pour l'IA on parle d'hiver de l'IA, la recherche s'effondre suite à désillusion scientifique. Cette crise a duré une décennie, dès les années 1970 l'intelligence artificielle prend un nouveau tournant. Avant : les méthodes sont des améliorations du combinatoire : on restreint l'énumération exhaustive à l'aide du bon sens, de fonctions d'évaluations et d'heuristiques. Après : les chercheurs sont convaincus que les programmes doivent avoir une connaissance approfondie du domaine étudié. Un programme d'IA doit avoir toute la connaissance nécessaire et ne l'utiliser qu'à bon escient. Ces idées ont conduit à la réalisation et à l'étude des systèmes experts ou systèmes à base de connaissances [13].

Les années 1980 et 1990 : Le début des années 1980 vit la première renaissance avec les systèmes experts, mais cette vague n'a pas duré longtemps. La fin des années 80, l'IA subit une nouvelle crise avec l'émergence des nouveaux ordinateurs personnels [2].

Pendant les années 1990, l'IA réintégré des labos de recherche et le début du « big data » cela est dû grâce à l'augmentation importante des capacités de calcul des machines. D'autre part, les réseaux de neurones et le deep Learning ont connu un premier essor au milieu des années 1990 [2].

Renaissance moderne : À partir des années 2010, l'IA connaît une révolution grâce à l'apprentissage profond (deep Learning), alimenté par l'augmentation des données disponibles et des avancées en matière de puissance de calcul. Des modèles comme les réseaux de neurones profonds permettent des avancées significatives dans des domaines tels que la vision par ordinateur et le traitement du langage naturel.

Applications contemporaines : Aujourd'hui, l'IA est omniprésente dans divers secteurs, allant des recommandations de contenu sur les plateformes de streaming aux systèmes de conduite autonome, illustrant son évolution rapide et son intégration dans la vie quotidienne.

2.2.2 Amélioration de l'intelligence artificielle

2.2.2.1 L'IA Faible (ou étroite, ANI)

L'intelligence artificielle faible, également connue sous le nom d'intelligence artificielle étroite (ANI), désigne des systèmes conçus pour effectuer des tâches spécifiques avec une performance souvent supérieure à celle des humains dans ces domaines limités. Ces systèmes sont programmés pour résoudre des problèmes précis, comme la reconnaissance vocale, la recommandation de produits ou le diagnostic médical, mais ne possèdent pas de compréhension générale ou de conscience. L'IA faible est omniprésente dans notre quotidien, intégrée dans des applications telles que les assistants virtuels, les chatbots et les systèmes de filtrage de spam, et bien qu'elle soit très efficace, elle ne peut pas fonctionner en dehors de son domaine d'application défini.

2.2.2.2 L'IA Forte (ou générale, AGI)

L'intelligence artificielle forte, ou intelligence générale artificielle (AGI), fait référence à des systèmes capables d'effectuer toute tâche intellectuelle qu'un humain peut accomplir. Contrairement à l'IA faible, l'AGI possède une compréhension contextuelle et peut apprendre, raisonner et s'adapter à de nouvelles situations sans intervention humaine. L'AGI vise à reproduire les capacités cognitives humaines, ce qui soulève des questions éthiques et philosophiques sur la conscience, la responsabilité et l'impact de ces systèmes sur la société. Bien que l'AGI reste largement théorique et n'ait pas encore été réalisée, elle représente un objectif recherché dans le domaine de la recherche en intelligence artificielle.

2.2.2.3 La super IA (ASI_ artificial super intelligent)

La super intelligence artificielle (ASI) désigne une forme d'intelligence qui dépasse de loin les capacités humaines dans presque tous les domaines, y compris la créativité, la résolution de problèmes et l'intelligence émotionnelle. L'ASI pourrait potentiellement révolutionner la société en apportant des solutions à des défis complexes tels que le changement climatique, les maladies et la pauvreté. Cependant, ce concept soulève des préoccupations majeures en matière de sécurité et d'éthique, car une entité dotée d'une intelligence supérieure pourrait agir de manière imprévisible et avoir un impact profond sur l'humanité. La recherche sur l'ASI implique donc des discussions sur la manière de garantir que son développement soit aligné avec les valeurs humaines et serve le bien commun.

2.2.3 La nécessité des réseaux de neurones en IA

Les réseaux de neurones occupent une place centrale dans l'intelligence artificielle (IA) moderne, car ils répondent à des défis que les méthodes algorithmiques traditionnelles ne peuvent surmonter. Pour bien comprendre leur rôle, il est essentiel de distinguer les concepts d'IA, de Machine Learning (ML) et de Deep Learning (DL).

- ***L'intelligence artificielle (IA) :*** est un domaine vaste visant à créer des systèmes capables d'imiter l'intelligence humaine (raisonnement, prise de décision, perception).

- **Le Machine Learning (ML)** : sous-ensemble de l'IA, se focalise sur des algorithmes qui « apprennent » à partir de données pour effectuer des tâches sans programmation explicite.
- **Le Deep Learning (DL)** : une branche avancée du ML, utilise des réseaux de neurones artificiels multicouches pour extraire des caractéristiques complexes et résoudre des problèmes hautement non linéaires (ex: reconnaissance faciale, traduction automatique).

Les réseaux de neurones, au cœur du Deep Learning (DL), surpassent les méthodes classiques de Machine Learning (ML) en modélisant des relations non linéaires et hiérarchiques dans des données complexes (images, textes). Contrairement aux modèles ML traditionnels (arbres de décision, SVM), ils évitent l'extraction manuelle de caractéristiques : un CNN, par exemple, analyse automatiquement les pixels d'une image couche par couche. Leur adaptabilité aux environnements dynamiques (véhicules autonomes) et leur capacité à généraliser malgré des données bruitées, couplées à la puissance de calcul moderne (GPU, cloud), permettent de résoudre des problèmes à haute dimensionnalité (traduction, diagnostic médical). En automatisant la conception de modèles, ils réduisent la dépendance à l'expertise humaine et incarnent l'évolution vers des systèmes autonomes, justifiant leur exploration détaillée dans la section suivante.

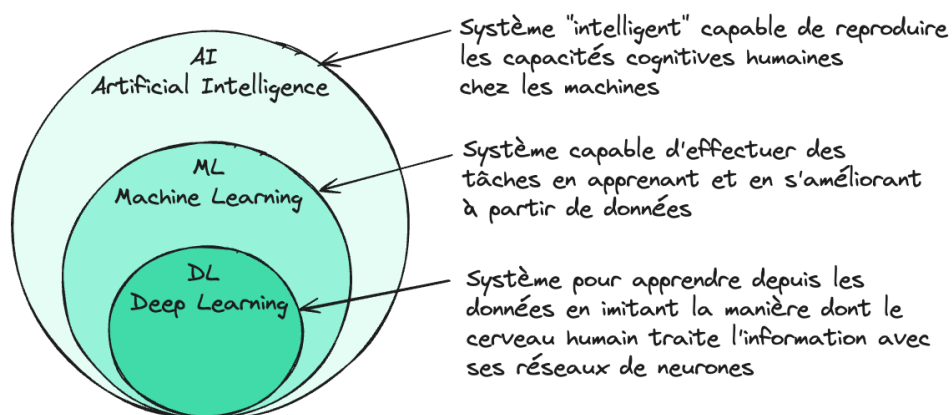


Figure 5: Difference entre IA, Machine Learning, Deep Learning

2.3 Réseaux de neurones

Un réseau de neurones peut être considéré comme un modèle mathématique de traitement distribué. Il se compose de plusieurs éléments de calcul non linéaires (neurones), qui fonctionnent en parallèle et sont reliés entre eux par des poids. Un réseau de neurones artificiels est un réseau fortement connecté de processeurs de base fonctionnant en parallèle. Chaque processeur de base calcule une sortie unique sur la base des informations reçues. Les neurones artificiels sont généralement utilisés sous forme de réseaux, qui varient selon les types de connexions entre neurones, et une cinquantaine de types peuvent être dénombrés. A titre d'exemples, nous citons: Rosembat perceptron, réseau Hopfield, etc.... Ce dernier est le plus utilisé dans le domaine de la modélisation et du contrôle de processus. Ils sont composés d'un nombre limité de neurones, qui sont disposés en couches. Les neurones de deux couches adjacentes sont reliés les uns aux autres en poids. Les informations contenues dans le réseau se propagent d'une couche à une autre, et elles sont appelées types "feedforward". On distingue trois types de couches:

Couche d'entrée: les neurones de cette couche reçoivent des valeurs d'entrée du réseau et les transmettent aux neurones cachés. Chaque neurone reçoit une valeur, donc la somme n'est pas ajoutée.

Couche cachée: chaque neurone de cette couche reçoit des informations des couches précédentes, effectue une sommation pondérée, puis la transforme en fonction de sa fonction d'activation (généralement une fonction sigmoïde). Ensuite, il envoie cette réponse aux neurones de la couche suivante.

Couche de sortie: elle a le même rôle que la couche cachée, La seule différence entre ces deux types de couches est que la sortie du neurone dans la couche de sortie n'est liée à aucun autre neurone. [15]

2.3.1 Principes des neurones artificiels

Chaque neurone artificiel est un processeur de base. Il reçoit un nombre variable d'entrées de neurones ou de capteurs en amont (qui font partie de la machine). Chacune de ses entrées est associée à un poids représentant la force de la connexion. Chaque processeur de base a une sortie, qui est ensuite sortie vers un nombre différent de neurones en aval. Chaque connexion est associée à un poids. Comme le montre la figure 6, il est très pratique de dessiner un diagramme neuronal. Cette représentation a été le point de départ de la

première vague d'attention à la génération de neurones formels de 1940 à 1970 (Mcculloch et al., 1943) (Minsky et al., 1969).

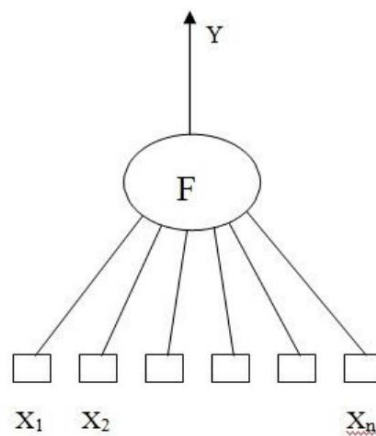


Figure 6: Neurone artificiel

Ensuite, le neurone effectue trois opérations sur son entrée:

- Poids: multipliez chaque entrée par un paramètre appelé poids de connexion,
- Sum: additionner l'entrée pondérée
- Activation: Passez cette somme à une fonction, qui s'appelle la fonction d'activation. [15]

2.3.2 Fonction d'activation

La fonction d'activation est une fonction mathématique responsable du type de l'information émise par le neurone, elle intervient après le calcul de la somme pondérée et l'ajout du biais, elle possède un influe profond sur les performances du réseau. D' où il est préférable de bien choisir le type de cette fonction dans chaque couche du réseau, notons que les neurones d'entrée ne possède pas de fonction d'activation, ils utilisent la fonction identité.

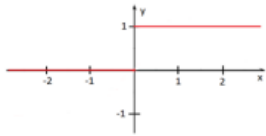
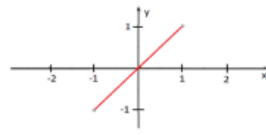
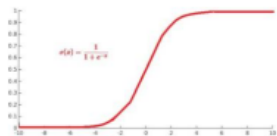
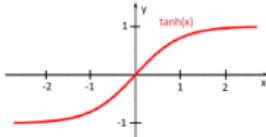
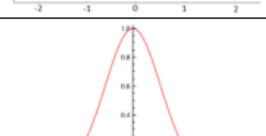
Fonction	Equation	Représentation graphique
Heaviside (Echelon)	$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$	
Rampe (Linéaire)	$f(x) = x$	
Sigmoïde	$\sigma(x) = \frac{1}{1 + e^{-x}}$	
Tangente hyperbolique	$f(x) = \tanh(x)$	
Unité de rectification linéaire (ReLU)	$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases}$	
Gaussienne	$f(x) = e^{-x^2}$	

Figure 7: Différents types de fonctions d'activations

2.3.3 Types de perceptron

Il s'agit du premier ARN opérationnel. C'est un réseau entièrement interconnecté avec seulement deux couches (entrée et sortie). Il est composé de neurones de seuil. Supervisez le processus d'apprentissage et augmentez progressivement son poids.

2.3.3.1 Le perceptron à une seule couche

Le perceptron monocouche est le premier modèle de neurones proposé créé. Le contenu de la mémoire locale du neurone se compose d'un vecteur de poids. Le calcul d'un perceptron d'une seule couche est effectué sur le calcul de la somme du vecteur d'entrée chacun avec la valeur multipliée par l'élément correspondant du vecteur des poids. La valeur qui est affichée dans la sortie sera l'entrée d'une fonction d'activation [15].

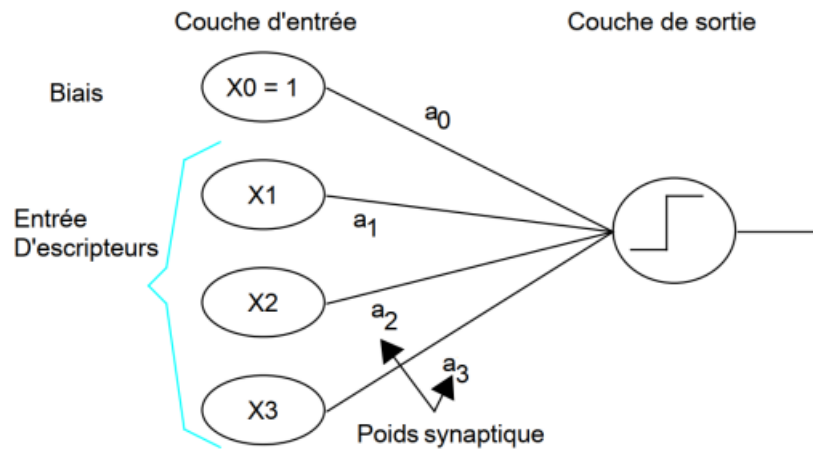


Figure 8: Perceptron simple

2.3.3.2 Perceptron multicouches (MLP)

Est un type de réseau de neurone Feed-Forward qui se caractérise par une couche d'entrée, un certain nombre de couches intermédiaires et une couche de sortie, qui sont entièrement connectées. Un MLP utilise back-propagation pour la formation. Le terme Feed-Forward se réfère à l'architecture en couches dans le réseau, en particulier qu'il n'y a pas de cycles dans le réseau. La structure de la couche garantit qu'il n'existe aucun cycle, car les couches ne sont autorisées à avoir des poids que de la couche directement précédente. Le terme entièrement connecté se réfère au fait que dans MLP, tous les nœuds dans une couche donnée ont un poids à tous les nœuds dans la couche précédente [16].

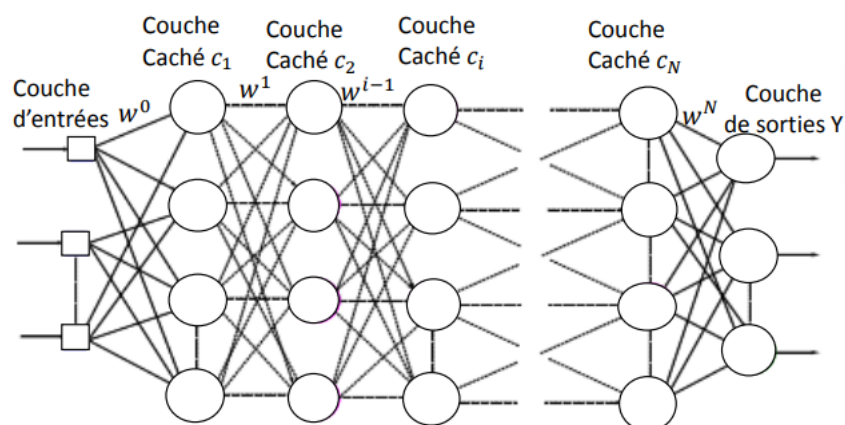


Figure 9: Perceptron multicouches (MLP)

2.3.4 Mécanismes d'apprentissage

2.3.4.1 Les types d'apprentissage

L'intelligence artificielle représente un domaine de l'informatique qui s'efforce de développer des systèmes capables d'exécuter des tâches habituellement réservées à l'intelligence humaine. Ces systèmes sont conçus pour gérer diverses fonctions comme l'apprentissage automatique, l'interprétation du langage humain, l'analyse d'images, la reconnaissance vocale, la résolution de problèmes complexes et la prise de décisions autonomes. Pour atteindre ces objectifs, l'intelligence artificielle s'appuie sur plusieurs approches techniques, dont Trois grandes approches relèvent de l'apprentissage automatique: L'apprentissage supervisé, L'apprentissage non supervisé, L'apprentissage par renforcement.

Apprentissage supervisé :

L'apprentissage supervisé consiste en la conception d'un modèle reliant des données d'apprentissage à un ensemble de valeurs de sortie. C'est-à-dire que les données d'entraînement qu'on fournit à l'algorithme comportent les solutions désirées, appelées étiquettes (en anglais, labels). Cette méthode permet donc à l'algorithme d'apprendre en comparant sa sortie réelle avec les sorties enseignées, afin de trouver les erreurs et modifier le modèle en conséquent. L'apprentissage supervisé confère au modèle la possibilité de prédire des valeurs d'étiquette sur des données non étiquetées supplémentaires [15].

Les algorithmes d'apprentissage supervisé sont les plus couramment utilisés, il y a deux types d'apprentissage supervisé:

- **La classification** : elle consiste à trouver le lien entre une variable d'entrée (X) et une variable de sortie discrète (Y), en suivant une loi multinomiale.
- **La régression** : elle consiste à prédire une valeur continue pour la variable de sortie.

Les algorithmes les plus célèbres utilisés dans cette approche sont les suivants :

- **SVM (Machines à vecteurs de support)** : est un apprentissage automatique très puissant et polyvalent modèle, capable d'effectuer la classification linéaire ou non linéaire, la régression, et même détection des valeurs aberrantes.

- **Les arbres de décision** : un arbre de décision sert à classer les futures observations, sachant qu'un corpus d'observations est déjà étiqueté.
- **Les réseaux de neurones** : c'est un modèle informatique qui partage certaines propriétés avec le cerveau humain, dans lequel de nombreuses unités simples travaillent en parallèle sans centralisation, ils permettent de trouver des patterns complexes dans les données, il se compose de valeurs d'entrées, poids, fonction de transfert et une valeur de sortie [16].

Apprentissage non supervisée :

Le ML non supervisé définit essentiellement toute méthode de ML qui tente d'apprendre la structure en l'absence d'une sortie identifiée [17]. Les ordinateurs apprennent à identifier par eux-mêmes les similitudes et les distinctions entre les données, et à regrouper ceux qui partagent des caractéristiques communes sans aucune orientation humaine stricte et rigoureuse [18]. L'apprentissage non supervisé fonctionne comme un enfant découvrant seul les fruits : il apprend à les distinguer en observant leurs formes et couleurs caractéristiques, sans qu'on lui ait préalablement enseigné leurs noms.

Apprentissage par renforcement :

Il existe beaucoup de problèmes où les sorties désirées que l'apprentissage supervisé exige sont difficile à spécifier. On ne dispose souvent que d'une information qualitative permettant l'évaluation de la réponse calculée [19]. L'apprentissage par renforcement utilise cette évaluation pour améliorer les performances du système. Cette évaluation peut être négative ou positive en fonction de l'attendu au travers de l'action. Ce type d'apprentissage est particulièrement adapté pour des tâches complexes et dynamiques, comme le contrôle de robots ou la prise de décision dans des jeux vidéo.

2.3.4.2 Règles d'apprentissage

La vitesse à laquelle les ANN apprennent dépend de plusieurs facteurs contrôlables (le temps, la complexité du réseau, sa taille, le choix du paradigme, l'architecture, le type de règle d'apprentissage, et la précision souhaitée). Ces facteurs jouent un rôle important dans la détermination du temps nécessaire pour former un réseau. La modification de l'un de ces facteurs peut soit allonger le temps de formation à une longueur déraisonnable, soit même aboutir à une précision inacceptable.

Il est évident qu'un rythme plus lent signifie que l'on passe beaucoup plus de temps à effectuer l'apprentissage pour produire un système correctement formé. Cependant, avec des taux d'apprentissage plus rapides, le réseau peut ne pas être en mesure de faire les discriminations fines possibles avec un système qui apprend plus lentement.

La plupart des fonctions d'apprentissage prévoient un certain taux d'apprentissage. Ce terme est généralement positif et se situe entre zéro et un. Si le taux d'apprentissage est supérieur à un, il est facile pour l'algorithme d'apprentissage de dépasser la correction des poids, et le réseau oscille. De petites valeurs du taux d'apprentissage ne corrigeront pas l'erreur actuelle aussi rapidement, mais si de petites mesures sont prises pour corriger les erreurs, il y a de bonnes chances d'arriver à la meilleure convergence minimale.

Loi de Hebb : La première, et sans doute la plus connue, règle d'apprentissage a été introduite par Donald Hebb. La description est apparue dans son livre *The Organisation of Behavior* en 1949. Sa règle de base est la suivante : Si un neurone reçoit une entrée d'un autre neurone, et si les deux sont très actifs (ont mathématiquement le même signe), le poids entre les neurones doit être renforcé.

$$\text{loi de hebb: } \Delta W_{ij} = R a_i a_j$$

Loi de Hopfield : Elle est similaire à la règle de Hebb, à l'exception du fait qu'elle précise l'ampleur du renforcement ou de l'affaiblissement. Elle stipule que si la sortie et l'entrée souhaitées sont toutes deux actives ou toutes deux inactives, il faut augmenter le poids de connexion.

$$\text{loi de hpfield: } \Delta W_{ij} = R a_i a_j$$

Loi Delta rule (Règle de Widrow-Hoff) : Cette règle est une autre variante de la règle de Hebb. Basée sur l'idée simple en modifiant continuellement les forces des connexions (poids synaptiques) d'entrée pour réduire la différence (le delta) entre la valeur de sortie souhaitée et la sortie réelle d'un élément de traitement.

$$\text{Widrow - Hoff: } \Delta W_{ij} = R a_i (a_j - W_{ij})$$

2.4 Apprentissage profond (Deep Learning)

À présent que nous avons précisé comment fonctionnent les réseaux de neurones de manière générale, nous allons aborder le domaine du Deep Learning. Cette famille

d'algorithmes a permis de faire des progrès importants dans les domaines de la classification des images et du traitement du langage par exemple. Les modèles de Deep Learning sont bâtis sur le même modèle que les perceptrons multicouches précédemment décrits. Cependant, il convient de souligner que les différentes couches intermédiaires sont plus nombreuses. Chacune des couches intermédiaires va être subdivisée en sous-partie, traitant un sous problème, plus simple et fournissant le résultat à la couche suivante, et ainsi de suite [22].

2.4.1 Notions de base du Deep Learning

2.4.1.1 Définition

L'apprentissage profond plus connu sous le nom « Deep Learning » regroupe un ensemble de techniques d'apprentissage automatique basées sur des approches mathématiques. Ces techniques sont utilisées pour modéliser des données et améliorer la compréhension de celle-ci. Le machine Learning a été développé dans les années 1950 par Alan Turing. Le Deep Learning est une forme avancée d'apprentissage automatique, qui utilise un réseau de neurones artificiels interconnectés, similaires au cerveau humain, pour traiter et mémoriser des informations comme les êtres humains [23].

2.4.1.2 principe fondamentaux

Le principe de base de l'apprentissage profond est d'entraîner un réseau de neurones à partir d'un ensemble de données d'apprentissage afin qu'il puisse effectuer des tâches spécifiques, telles que la reconnaissance d'images, la classification de textes, la prédiction de séries temporelles, etc. et Pour résoudre des problèmes de la meilleure façon possible. Ce processus est effectué à l'aide d'un algorithme d'optimisation qui cherche à minimiser une fonction de perte ou de coût définie pour la tâche en question. Une fois que le réseau de neurones a été entraîné, il peut être utilisé pour effectuer des prédictions sur de nouvelles données en général plus ou moins similaires à celles utilisées pour l'entraînement (figure 7)., il offre des moyens d'apprendre les représentations de données de manière supervisée et non Supervisée à l'aide de la hiérarchie des couches, qui permet un traitement multiple, et par rapport au méthodes conventionnelles de ML.

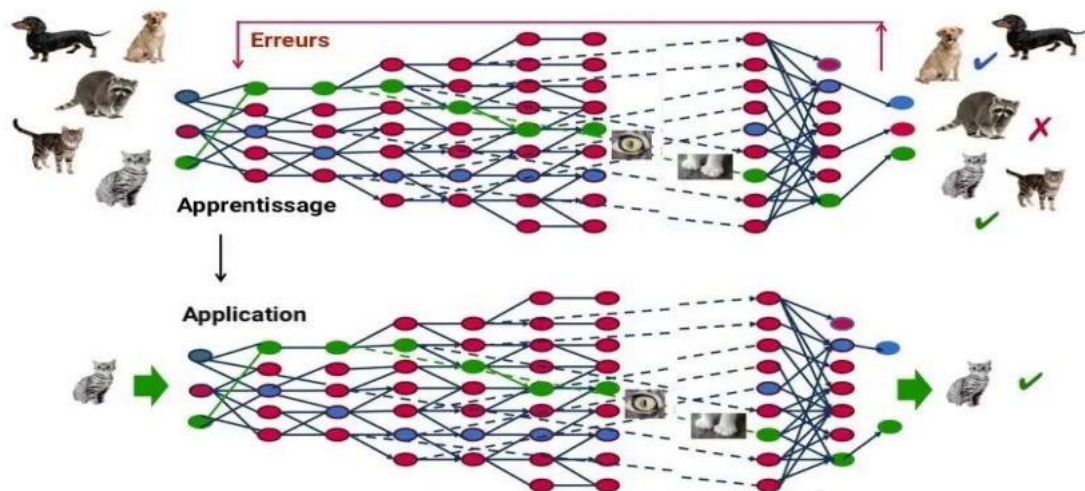


Figure 10: Exemple d'apprentissage profond

2.4.2 Comment fonctionne le deep Learning ?

Comme à l'intérieur du cerveau humain, les signaux voyagent entre les neurones du cerveau artificiel. Le secret de cette prouesse repose en grande partie sur les algorithmes. Dans le cas de la reconnaissance visuelle, pour être performant, l'algorithme du deep Learning doit être capable d'identifier toutes les formes existantes et dans tous les angles.

Ainsi, il sera capable de détecter une voiture sur la route au milieu du paysage. Ceci n'est possible que si la machine a suivi un entraînement poussé. Et ceci passe par la visualisation de milliers de photographies sur lesquelles apparaissent une voiture, de toutes les formes et dans tous les angles possibles. Lorsque l'image nouvelle apparaît, elle est envoyée au réseau de neurones qui se charge de les analyser et de déterminer si l'objet au milieu du cliché est bel et bien une voiture. La machine a gagné son pari ? Elle garde sa bonne réponse au chaud, car elle l'aidera à résoudre d'autres situations similaires le jour où elle devra reconnaître une autre voiture. [23]

2.4.3 Quelques algorithmes de deep Learning

Il existe différents algorithmes de Deep Learning. Nous pouvons ainsi citer :

- **Les réseaux de neurones profonds (Deep Neural Networks) :** Ces réseaux sont similaires aux réseaux MLP mais avec plus de couches cachées. L'augmentation du nombre de couches, permet à un réseau de neurones de détecter de légères

variations du modèle d'apprentissage, favorisant le sur-apprentissage ou sur-ajustement (« overfitting »).

- **Les réseaux de neurones convolutionnels (CNN ou Convolutional Neural Networks)**

: Le problème est divisé en sous parties, et pour chaque partie, un «cluster» de neurones sera créer afin d'étudier cette portion spécifique. Par exemple, pour une image en couleur, il est possible de diviser l'image sur la largeur, la hauteur et la profondeur (les couleurs).

- **La machine de Boltzmann profonde (Deep Belief Network)** : Ces algorithmes fonctionnent suivant une première phase non supervisée, suivi de l'entraînement classique supervisé. Cette étape d'apprentissage non-supervisée, permet, en outre, de faciliter l'apprentissage supervisé.

2.5 Les réseaux de neurones convolutionnels (CNN)

Les réseaux de neurones convolutionnels sont à ce jour les modèles les plus performants pour classer des images. Désignés par l'acronyme CNN, de l'anglais Convolutional Neural Network, ils comportent deux parties bien distinctes. En entrée, une image est fournie sous la forme d'une matrice de pixels. Elle a 2 dimensions pour une image en niveaux de gris. La couleur est représentée par une troisième dimension, de profondeur 3 pour représenter les couleurs fondamentales [Rouge, Vert, Bleu]. La première partie d'un CNN est la partie convolutive à proprement parler. Elle fonctionne comme un extracteur de caractéristiques des images. Une image est passée à travers une succession de filtres, ou noyaux de convolution, créant de nouvelles images appelées cartes de convolutions. Certains filtres intermédiaires réduisent la résolution de l'image par une opération de maximum local. Au final, les cartes de convolutions sont mises à plat et concaténées en un vecteur de caractéristiques, appelé code CNN [24].

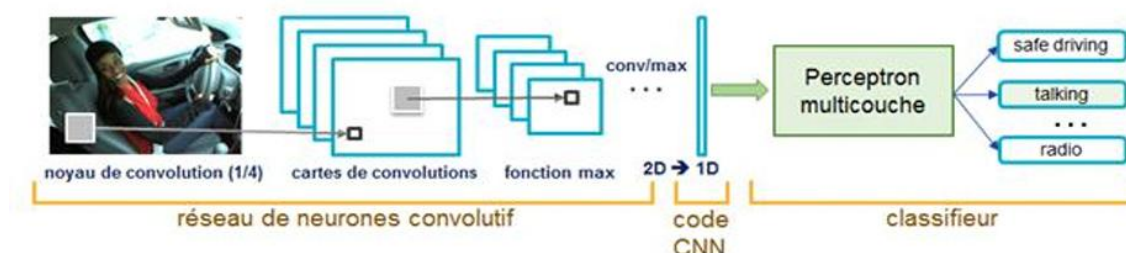


Figure 11: Architecture standard d'un réseau de neurone convolutionnel

Ce code CNN en sortie de la partie convolutive est ensuite branché en entrée d'une deuxième partie, constituée de couches entièrement connectées (perceptron multicouche). Le rôle de cette partie est de combiner les caractéristiques du code CNN pour classer l'image. La sortie est une dernière couche comportant un neurone par catégorie. Les valeurs numériques obtenues sont généralement normalisées entre 0 et 1, de somme 1, pour produire une distribution de probabilité sur les catégories.

2.5.1 Architecture de réseaux de neurone convolutionnel :

Les réseaux de neurones convolutionnels sont basés sur le perceptron multicouche (MLP), et inspirés du comportement du cortex visuel des vertébrés. Bien qu'efficaces pour le traitement d'images, les MLP ont beaucoup de mal à gérer des images de grande taille, ce qui est dû à la croissance exponentielle du nombre de connexions avec la taille de l'image. Par exemple, si on prend une image de taille $32 \times 32 \times 3$ (32 de large, 32 de haut, 3 canaux de couleur), un seul neurone entièrement connecté dans la première couche cachée du MLP aurait 3072 entrées ($32 \times 32 \times 3$). Une image 200×200 conduirait ainsi à traiter 120 000 entrées par neurone ce qui, multiplié par le nombre de neurones, devient énorme.

Les CNN visent à limiter le nombre d'entrées tout en conservant la forte corrélation « spatialement locale » des images naturelles. Par opposition aux MLP, les CNN ont les traits distinctifs suivants :

1. 'Volumés 3D de neurones'. La couche de neurones n'est plus simplement une surface (perceptron), mais devient un volume avec une profondeur. Si on considère un seul champ récepteur du CNN, les n neurones associés (sur la profondeur) forment l'équivalent de la première couche d'un MLP.

2. 'Connectivité locale'. Grâce au champ récepteur qui limite le nombre d'entrées du neurone, tout en conservant l'architecture MLP, les CNN assurent ainsi que les « filtres » produisent la réponse la plus forte à un motif d'entrée spatialement localisé, ce qui conduit à une représentation parcimonieuse de l'entrée. Une telle représentation occupe moins d'espace en mémoire. De plus, le nombre de paramètres à estimer étant réduit, leur estimation (statistique) est plus robuste pour un volume de données fixé (comparé à un MLP).

3. 'Poids partagés' : Dans les CNNs, les paramètres de filtrage d'un neurone (pour un champ récepteur donné) sont identiques pour tous les autres neurones d'un même noyau (traitant tous les autres champs récepteurs de l'image). Ce paramétrage (vecteur de poids et biais) est défini dans une « carte de fonction ». Cela signifie que tous les neurones dans une couche de convolution donnée détectent exactement la même caractéristique. En multipliant les champs récepteurs, il devient possible de détecter des éléments indépendamment de leur position dans le champ visuel, ce qui induit une propriété d'invariance par translation.

Ensemble, ces propriétés permettent aux réseaux de neurones convolutionnels d'obtenir une meilleure généralisation (en termes d'apprentissage) sur des problèmes de vision. Le partage de poids permet aussi de réduire considérablement le nombre de paramètres libres à apprendre, et ainsi les besoins en mémoire pour le fonctionnement du réseau. La diminution de l'empreinte mémoire permet l'apprentissage de réseaux plus grands donc souvent plus puissants. [24]

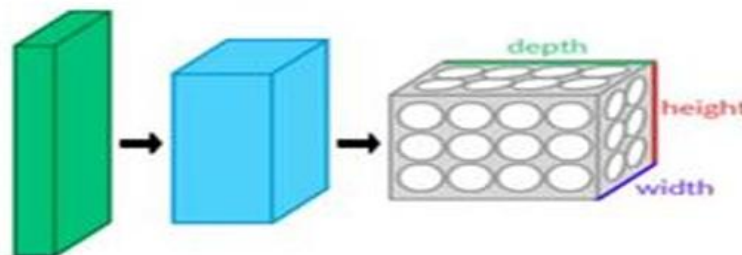


Figure 12: Une couche du CNN en 3 dimensions. (Vert = volume d'entrée, bleu = volume du champ récepteur, gris = couche de CNN, cercles = neurones artificiels indépendants)

Une architecture CNN est formée par un empilement de couches de traitement indépendantes :

- La couche de convolution (CONV) qui traite les données d'un champ récepteur.
- La couche de pooling (POOL), qui permet de compresser l'information en réduisant la taille de l'image intermédiaire (souvent par sous-échantillonnage).
- La couche de correction (Relu), souvent appelée par abus 'RELU' en référence à la fonction d'activation (Unité de rectification linéaire).

- La couche "entièrement connectée" (FC), qui est une couche de type perceptron.
- La couche de perte (LOSS).

2.5.1.1 Couche de convolution(CONV) :

La couche de convolution est le bloc de construction de base d'un CNN. Trois paramètres permettent de dimensionner le volume de la couche de convolution la profondeur, **le pas** et **la marge**.

1. **Profondeur de la couche** : nombre de noyaux de convolution (ou nombre de neurones associés à un même champ récepteur).
2. **Le pas**: contrôle le chevauchement des champs récepteurs. Plus le pas est petit, plus les champs récepteurs se chevauchent et plus le volume de sortie sera grand.
3. **La marge (à 0) ou zero padding** : parfois, il est commode de mettre des zéros à la frontière du volume d'entrée. La taille de ce 'zero-padding' est le troisième hyperparamètre. Cette marge permet de contrôler la dimension spatiale du volume de sortie. En particulier, il est parfois souhaitable de conserver la même surface que celle du volume d'entrée.

Si le pas et la marge appliquée à l'image d'entrée permettent de contrôler le nombre de champs récepteurs à gérer (surface de traitement), la profondeur permet d'avoir une notion de volume de sortie, et de la même manière qu'une image peut avoir un volume, si on prend une profondeur de 3 pour les trois canaux RGB d'une image couleur, la couche de convolution va également présenter en sortie une profondeur. C'est pour cela que l'on parle plutôt de "volume de sortie" et de "volume d'entrée", car l'entrée d'une couche de convolution peut être soit une image soit la sortie d'une autre couche de convolution. La taille spatiale du volume de sortie peut être calculée en fonction de la taille du volume d'entrée W_i la surface de traitement K (nombre de champs récepteurs), le pas S avec lequel ils sont appliqués, et la taille de la marge P La formule pour calculer le nombre de neurones du volume de sortie est :

$$W_0 = \frac{w_i - k + 2P}{s} + 1$$

Si W_0 n'est pas entier, les neurones périphériques n'auront pas autant d'entrée que les autres. Il faudra donc augmenter la taille de la marge (pour recréer des entrées virtuelles). Souvent, on considère un pas $S=1$, on calcule donc la marge de la manière suivante :

$P = \frac{k-1}{2}$ Si on souhaite un volume de sortie de même taille que le volume d'entrée. Dans ce cas particulier la couche est dite "connectée localement".

2.5.1.2 Couche de pooling (POOL) :

Un autre concept important des CNNs est le pooling, ce qui est une forme de sous-échantillonnage de l'image. L'image d'entrée est découpée en une série de rectangles de n pixels de côté ne se chevauchant pas (pooling). Chaque rectangle peut être vu comme une tuile.

Le signal en sortie de tuile est défini en fonction des valeurs prises par les différents pixels de la tuile. Le pooling réduit la taille spatiale d'une image intermédiaire, réduisant ainsi la quantité de paramètres et de calcul dans le réseau. Il est donc fréquent d'insérer périodiquement une couche de pooling entre deux couches convolutives successives d'une architecture CNN pour contrôler l'overfitting (sur-apprentissage). L'opération de pooling créait aussi une forme d'invariance par translation.

La couche de pooling fonctionne indépendamment sur chaque tranche de profondeur de l'entrée et la redimensionne uniquement au niveau de la surface. La forme la plus courante est une couche de mise en commun avec des tuiles de taille 2×2 (largeur/hauteur) et comme valeur de sortie la valeur maximale en entrée. On parle dans ce cas de « Max-Pool 2×2 ».

Il est possible d'utiliser d'autres fonctions de pooling que le maximum. On peut utiliser un « average pooling » (la sortie est la moyenne des valeurs du patch d'entrée), du « L2-norm pooling ». Dans les faits, même si initialement l'average pooling était souvent utilisé il s'est avéré que le max-pooling était plus efficace car celui-ci augmente plus significativement l'importance des activations fortes. En d'autres circonstances, on pourra utiliser un pooling stochastique.

Le pooling permet de gros gains en puissance de calcul. Cependant, en raison de la réduction agressive de la taille de la représentation (et donc de la perte d'information associée), la tendance actuelle est d'utiliser de petits filtres (type 2×2). Il est aussi possible d'éviter la couche de pooling mais cela implique un risque sur-apprentissage plus important.

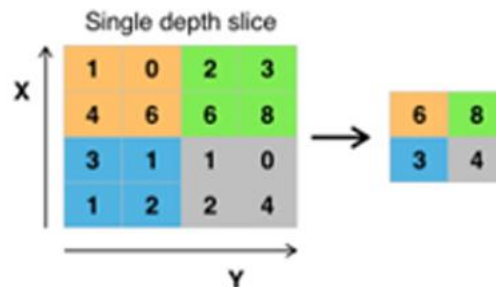


Figure 13 : Pooling avec un filtre 2x2 et un pas de 2

2.5.1.3 Couches de correction (RELU) :

Il est possible d'améliorer l'efficacité du traitement en intercalant entre les couches de traitement une couche qui va opérer une fonction mathématique (fonction d'activation) sur les signaux de sortie. La fonction Relu (abréviation de Unités Rectifié linéaires) : $F(x) = \max(0, x)$ Cette fonction force les neurones à retourner des valeurs positives.

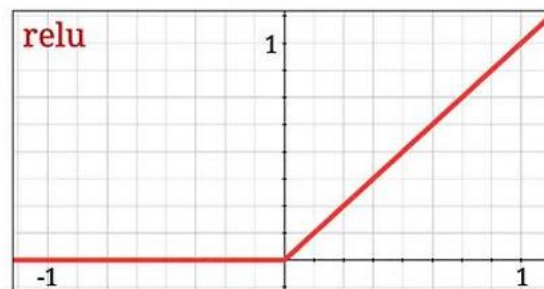


Figure 14: Fonction RELU

2.5.1.4 Couche entièrement connectée (FC) :

Après plusieurs couches de convolution et de max-pooling, le raisonnement de haut niveau dans le réseau neuronal se fait via des couches entièrement connectées. Les neurones dans une couche entièrement connectée ont des connexions vers toutes les sorties de la couche précédente. Leurs fonctions d'activations peuvent donc être calculées avec une multiplication matricielle suivie d'un décalage de polarisation.

2.5.1.5 Couche de perte (LOSS) :

La couche de perte spécifie comment l'entraînement du réseau pénalise l'écart entre le signal prévu et réel. Elle est normalement la dernière couche dans le réseau. Diverses

fonctions de perte adaptées à différentes tâches peuvent y être utilisées. La fonction « Softmax » permet de calculer la distribution de probabilités sur les classes de sortie.

Softmax :

Le principe de softmax est simple : on part d'un vecteur $(x_0, x_1, \dots)$ avec des coefficients quelconques, et on veut obtenir un vecteur avec des coefficients positifs de somme 1. Il suffit alors :

- De remplacer chaque x_i par $\exp(x_i)$ pour obtenir un nombre positif.
- Puis de diviser par la somme des coefficients pour le normaliser le résultat.

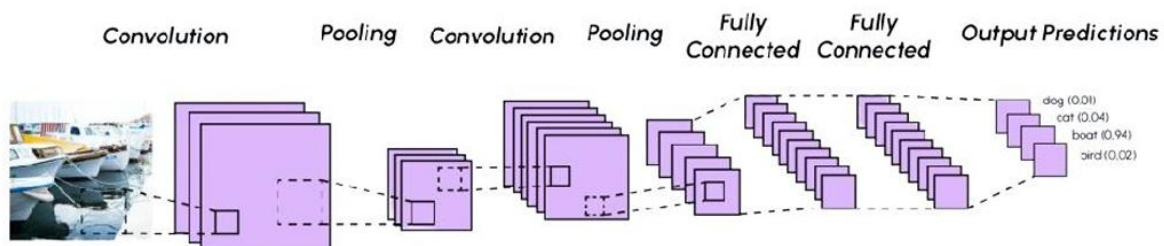
Ce qui donne comme formule $\text{softmax}(x_0, x_1, \dots) = (Y_0, Y_1, \dots)$ où les Y_i valent :

$$y_i = \frac{\exp(x_i)}{\sum_{p \geq 0} \exp(x_p)}$$

2.5.2 Exemple de modèles de CNN

La forme la plus commune d'une architecture CNN empile quelques couches Conv-ReLU, les suit avec des couches Pool, et répète ce schéma jusqu'à ce que l'entrée soit réduite dans un espace d'une taille suffisamment petite. À un moment, il est fréquent de placer des couches entièrement connectées (FC). La dernière couche entièrement connectée est reliée vers la sortie. Voici quelques architectures communes CNN qui suivent ce modèle :

- INPUT -> CONV -> RELU -> FC
- INPUT -> [CONV -> RELU -> POOL] * 2 -> FC -> RELU -> FC Ici, il y a une couche de CONV unique entre chaque couche POOL
- INPUT -> [CONV -> RELU -> CONV -> RELU -> POOL] * 3 -> [FC -> RELU] * 2 -> FC Ici, il y a deux couches CONV empilées avant chaque couche POOL. L'empilage des couches CONV avec de petits filtres de pooling (plutôt un grand filtre) permet un traitement plus puissant, avec moins de paramètres. Cependant, avec l'inconvénient de demander plus de puissance de calcul (pour contenir tous les résultats intermédiaires de la couche CONV).



2.6 Choix des paramètres

Les CNNs utilisent plus de paramètres qu'un MLP standard. Même si les règles habituelles pour les taux d'apprentissage et des constantes de régularisation s'appliquent toujours, il faut prendre en considération les notions de nombre de filtres, leur forme et la forme du max pooling [24].

2.6.1 Nombre de filtres :

Comme la taille des images intermédiaires diminue avec la profondeur du traitement, les couches proches de l'entrée ont tendance à avoir moins de filtres tandis que les couches plus proches de la sortie peuvent en avoir davantage. Pour égaliser le calcul à chaque couche, le produit du nombre de caractéristiques et le nombre de pixels traités est généralement choisi pour être à peu près constant à travers les couches. Pour préserver l'information en entrée, il faudrait maintenir le nombre de sorties intermédiaires (nombre d'images intermédiaire multiplié par le nombre de positions de pixel) pour être croissante (au sens large) d'une couche à l'autre. le nombre d'images intermédiaires contrôle directement la puissance du système, dépend du nombre d'exemples disponibles et la complexité du traitement.

2.6.2 Forme du filtre :

Les formes de filtre varient grandement dans la littérature. Ils sont généralement choisis en fonction de l'ensemble de données. Les meilleurs résultats sur les images de MNIST (28x28) sont habituellement dans la gamme de 5x5 sur la première couche, tandis que les ensembles de données d'images naturelles (souvent avec des centaines de pixels dans chaque dimension) ont tendance à utiliser de plus grands filtres de première couche de 12x12, voire 15x15. Le défi est donc de trouver le bon niveau de granularité de manière à créer des abstractions à l'échelle appropriée et adaptée à chaque cas.

2.6.3 Forme du Max Pooling :

Les valeurs typiques sont 2x2. De très grands volumes d'entrée peuvent justifier un pooling 4x4 dans les premières couches. Cependant, le choix de formes plus grandes va considérablement réduire la dimension du signal, et peut entraîner la perte de trop d'information.

2.7 Méthodes de régularisation

Pour ne pas tomber dans le problème de sur apprentissage il y a des méthodes de régularisation à utiliser. Elle se répartit entre méthodes empiriques et explicites.

2.7.1 Empirique :

2.7.1.1 Dropout :

Les couches "FC" (Fully Connected) occupent la majeure partie de la mémoire du CNN. D'ailleurs le concept de FC crée un problème exponentiel de mémoire appelé "overfitting" ("sur-connexion" conduisant au sur-apprentissage) ralentissant le traitement de l'information. Pour prévenir cela, la méthode du dropout est utilisée pour "éteindre" les neurones aléatoirement (avec une probabilité prédéfinie, souvent un neurone sur deux) ainsi que les neurones périphériques. Ainsi, avec moins de neurones, le réseau est plus réactif et peut donc apprendre plus rapidement. À la fin de la séance d'apprentissage, les neurones "éteints" sont "rallumés" (avec leurs poids originaux). Plus la couche FC est proche de l'image source, moins on éteindra de neurones. L'objectif est d'éteindre et rallumer les neurones aléatoirement, dans le cadre d'entraînements successifs. Une fois les séries d'entraînements terminées, on rallume tous les neurones et on utilise le réseau comme d'habitude. Cette technique a montré non seulement un gain dans la vitesse d'apprentissage, mais en déconnectant les neurones, on a aussi limité des effets marginaux, rendant le réseau plus robuste et capable de mieux généraliser les concepts appris [24].

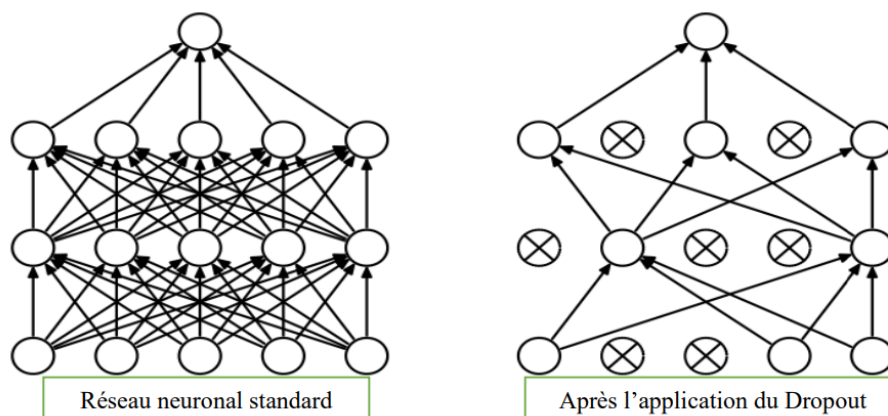


Figure 15: Modèle de réseau neuronal par Dropout [25]

2.7.1.2 DropConnect :

Le DropConnect est une évolution du dropout, où on ne va non plus éteindre un neurone, mais une connexion (l'équivalent de la synapse), et ce de manière toujours aléatoire. Les résultats sont similaires (rapidité, capacité de généralisation de l'apprentissage), mais présentent une différence au niveau de l'évolution des poids des connexions. Une couche FC avec un DropConnect peut s'apparenter à une couche à connexion "diffuse".

2.7.1.3 Pooling stochastique :

Le pooling stochastique, contrairement aux méthodes déterministes (comme le max ou l'average pooling), introduit une dimension aléatoire dans le processus de réduction. Cette approche permet au réseau d'explorer des variations subtiles dans les données et d'apprendre à partir d'un échantillonnage de la région locale, ce qui peut contribuer à améliorer la généralisation du modèle.

2.7.2 Explicite :

2.7.2.1 Taille du réseau :

Une méthode explicite de régularisation consiste à contrôler la taille du réseau – c'est-à-dire le nombre de couches et de neurones. En limitant cette taille, on réduit la capacité du modèle à mémoriser le bruit, ce qui force le réseau à se concentrer sur les caractéristiques dont l'information est réellement généralisable [25].

2.7.2.2 Dégradation du poids :

Le concept est de considérer le vecteur des poids d'un neurone (liste des poids associés aux signaux entrants), et de lui rajouter un vecteur d'erreur proportionnel à la somme des poids (norme 1) ou du carré des poids (norme 2 ou euclidienne). Ce vecteur d'erreur peut ensuite être multiplié par un coefficient de proportionnalité que l'on va augmenter pour pénaliser davantage les vecteurs de poids forts.

- La régularisation par norme 1 : La spécificité de cette régulation est de diminuer le poids des entrées aléatoires et faibles et d'augmenter le poids des entrées "importantes". Le système devient moins sensible au bruit.
- La régularisation par norme 2 : (norme euclidienne) La spécificité de cette régulation est de diminuer le poids des entrées fortes, et de forcer le neurone à plus prendre en compte les entrées de poids faible.

Les régularisations par norme 1 et norme 2 peuvent être combinées : c'est la "régularisation de réseau élastique" (Elastic net régulation).

2.8 Conclusion

Ce chapitre a posé les bases théoriques et techniques du Deep Learning, en insistant sur son rôle central dans l'IA moderne. Nous avons exploré les architectures de réseaux de neurones convolutifs (CNN), les paramètres clés de leur conception, et les méthodes pour optimiser leur performance tout en évitant le surajustement. Ces concepts préparent le terrain pour une application pratique, où la théorie rencontre les données réelles. Dans le chapitre suivant, nous plongerons dans l'implémentation concrète : utilisation d'un dataset d'images d'animaux (buffles, chats, chiens, oiseaux, zèbres, éléphants), analyse du code source dédié à la classification, et évaluation des résultats via une matrice de confusion. Cette étape cruciale permettra de mesurer l'efficacité des modèles CNN développés et de valider leur capacité à distinguer des espèces animales avec précision.

Chapitre 3 : Classification d'images – Implémentation

- *Introduction*
- *Prétraitement des données*
- *Description du dataset*
- *Logiciels et librairies utilisés dans
l'implémentation*
- *Architecture du modèle*
- *Entraînement du modèle*
- *Évaluation des performances*
- *Résultats obtenus et discussion*
- *Conclusion*

3.1 Introduction

Ce chapitre constitue la pierre angulaire de notre étude, matérialisant les concepts théoriques abordés précédemment par le développement concret d'un modèle de classification multiclasse d'images basé sur un réseau de neurones convolutifs (CNN). L'objectif est de concevoir, entraîner et évaluer un système capable de distinguer six espèces animales (Buffalo, chat, chien, éléphant, oiseau, zèbre) en exploitant des techniques avancées de traitement d'images.

Dans un premier temps, nous détaillerons les étapes de prétraitement des données, incluant la normalisation des pixels et l'augmentation artificielle des images (rotation, zoom). Ces techniques visent à optimiser la qualité des données d'entrée et à atténuer les risques de surapprentissage. Ensuite, nous décrirons la structure et les sources du dataset utilisé (Kaggle, Images.cv), en mettant l'accent sur la répartition équilibrée des classes et les stratégies adoptées pour gérer les déséquilibres résiduels.

La troisième partie abordera les Logiciels et bibliothèques Utilisés dans l'implémentation (Python, TensorFlow, Keras, Scikit-learn, etc.), ainsi que la configuration matérielle dédiée à l'entraînement du modèle. Nous expliquerons pourquoi une architecture CNN séquentielle a été privilégiée, en détaillant le rôle de chaque couche (convolution, pooling, dropout) et le choix des hyperparamètres (taux d'apprentissage, taille des lots).

Enfin, nous exposerons les résultats obtenus après entraînement, en analysant les métriques de performance (accuracy, F1-score) et la matrice de confusion. Une attention particulière sera portée aux erreurs récurrentes (ex : confusion entre Buffalo et éléphant) et à leur interprétation, afin d'identifier les limites du modèle et les pistes d'amélioration. Des visualisations, telles que les courbes d'apprentissage et des exemples d'images mal classées, viendront étayer cette analyse.

L'originalité de ce chapitre réside dans son approche holistique, alliant rigueur technique et analyse critique. Il ne s'agit pas seulement de présenter un modèle fonctionnel, mais aussi d'évaluer sa robustesse face à des défis réels : variabilité intra-classe, biais de contexte, et qualité hétérogène des images. Les conclusions tirées serviront de base pour des travaux futurs, notamment l'intégration de modèles pré-entraînés ou l'extension à des applications embarquées.

En résumé, ce chapitre structure la colonne vertébrale de notre étude, démontrant comment une architecture CNN personnalisée peut répondre à un problème complexe de classification, tout en soulignant l'importance d'une validation méthodique et transparente.

3.2 Prétraitement des données

3.2.1 Normalisation des images

Avant l'entraînement, toutes les images subissent une normalisation standard pour optimiser la stabilité numérique du modèle. Les valeurs des pixels, initialement comprises entre 0 et 255, sont redimensionnées à l'intervalle $[0, 1]$ via une division par 255 ($\text{rescale}=1./255$), réduisant ainsi la sensibilité du modèle aux variations d'éclairage. Cette opération est appliquée uniformément aux ensembles d'entraînement, de validation et de test. Par ailleurs, les images sont redimensionnées à une résolution fixe de 224×224 pixels pour homogénéiser les entrées du réseau, tout en conservant l'espace colorimétrique RGB afin de préserver les motifs chromatiques critiques (ex : rayures de zèbre).

3.2.2 Augmentation des données (rotation, zoom, etc.)

Pour améliorer la généralisation du modèle et lutter contre le surapprentissage, une augmentation dynamique des données est appliquée uniquement à l'ensemble d'entraînement via ImageDataGenerator de Keras. Cette technique génère des variantes synthétiques des images originales en temps réel pendant l'entraînement, sans surcoût de stockage. Les transformations incluent :

- ≠ **Rotation aléatoire** ($\pm 25^\circ$) pour simuler des angles de prise de vue variés.
- ≠ **Décalages horizontaux/verticaux** (20% de la largeur/hauteur) et cisaillement (20%) pour imiter des cadrages imparfaits.
- ≠ **Zoom aléatoire** (jusqu'à 20%) et retournement horizontal pour diversifier les perspectives.
- ≠ **Remplissage des bords** ($\text{fill_mode}='nearest'$) pour éviter les artefacts lors des transformations géométriques.

Exemple de configuration :

```
# Data Generators
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=25,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

Figure 16 : Exemple de Configuration

Ces perturbations artificielles, combinées à la normalisation, permettent au modèle d'utiliser différentes caractéristiques et variations réelles, car elles permettent un fonctionnement robuste dans des conditions réelles. Le sens validation/test, en révision, reste non augmenté pour assurer une évaluation complète des performances.

3.3 Description du dataset

3.3.1 Source et structure du dataset

Le dataset utilisé est une compilation personnalisée issue de sources variées, soigneusement sélectionnées pour garantir une diversité représentative des six classes animales étudiées. Les images de Buffalo proviennent principalement de la plateforme images.cv, complétées par des captures issues de recherches Google ciblant des contextes naturels variés (savanes, forêts). Les données pour les chats sont extraites d'un ensemble public disponible sur Kaggle, tandis que les images de chiens combinent des ressources de Kaggle (notamment l'ensemble "Dogs vs Cats") et de images.cv pour couvrir une plus grande variété de races. Les classes éléphant, oiseau et zèbre sont intégralement constituées à partir de images.cv, sélectionnées pour leur qualité et leur résolution homogène.

Au total, le dataset contient 17 389 images est divisé en Trois parties : l'ensemble d'entraînement, l'ensemble de validation et l'ensemble de test. Chaque ensemble comprend six classes équilibrées Buffalo, chat, chien, éléphant, oiseau, et zebra - avec des images réparties dans des sous-dossiers correspondants. L'ensemble d'entraînement contient des images utilisées pour entraîner le modèle, l'ensemble de validation sert à

ajuster les hyperparamètres et détecter le surapprentissage et l'ensemble de test est réservé à l'évaluation des performances post-entraînement. Chaque classe a un nombre égal d'images dans les ensembles. Cette organisation facilite l'entraînement et l'évaluation du modèle de classification d'images d'animaux.

3.3.2 Répartition des classes

3.3.2.1 L'ensemble d'entraînement (Train) :

Ce dossier contient des images utilisées pour entraîner notre modèle. Chaque classe d'animal a son propre sous-dossier contenant les images correspondantes. Par exemple :

- ≠ Buffalo = 1876 images
- ≠ Chat = 2737 images
- ≠ Chien = 2731 images
- ≠ Elephant = 2560 images
- ≠ Oiseau = 1750 images
- ≠ Zebra = 2056 images

3.3.2.2 l'ensemble de validation (Validation) :

L'ensemble de validation (Validation), constitué de 15 % des données (1931 images, environ 321 par classe), joue un rôle clé dans le réglage des hyperparamètres (comme l'arrêt anticipé) et la détection du surapprentissage. Ces images, non augmentées et simplement normalisées, servent de référence objective pendant l'entraînement :

- ≠ Buffalo = 329 images
- ≠ Chat = 300 images
- ≠ Chien = 316 images
- ≠ Elephant = 310 images
- ≠ Oiseau = 339 images
- ≠ Zebra = 337 images

3.3.2.3 L'ensemble de test (Test) :

Ce dossier contient un ensemble distinct d'images (1757 images, environ 300 images par classe) qui ne sont pas utilisées pendant l'entraînement mais sont réservées pour évaluer les performances du modèle après l'entraînement. Comme pour l'ensemble

d'entraînement, les images sont organisées par classe dans des sous dossiers correspondants.

3.3.3 Exemple des images

Buffalo



chat



oiseau



Chien



elephant



zébra



3.4 Logiciels et librairies utilisés dans l'implémentation

3.4.1 Langage utilise

3.4.1.1 Python



Figure 17: logo de python

Python est le langage de programmation open source le plus utilisé par les informaticiens. Ce langage s'est propulsé au sommet de la gestion d'infrastructure, de l'analyse de données ou du développement de logiciels. En fait, parmi les nombreuses fonctionnalités de Python, les développeurs peuvent se concentrer sur ce qu'ils font, plutôt que sur la façon dont ils le font. Il libère les développeurs des contraintes formelles qui tourmentaient

leur temps avec les langages plus anciens. Par conséquent, développer du code en Python est plus rapide que dans d'autres langages [28].

3.4.2 Bibliothèque Utilises

3.4.2.1 TensorFlow

TensorFlow, initialement développé par l'équipe Google Brain en 2011 sous le nom de DistBelief, est une bibliothèque open source dédiée au Machine Learning et au Deep Learning. Son origine remonte à un système propriétaire utilisé en interne chez Google pour des tâches complexes de traitement des données. En 2015, Google a rendu TensorFlow open source, permettant à la communauté de développeurs de l'utiliser librement.

TensorFlow se distingue par son approche graphique pour représenter les opérations mathématiques, où les calculs sont organisés sous forme de graphiques avec des nœuds représentant des opérations et des arêtes représentant les flux de données. Son API front-end est principalement basée sur Python, offrant ainsi une facilité de développement et d'expérimentation pour les chercheurs et les développeurs. L'exécution des opérations se fait en arrière-plan en C++, assurant des performances élevées.

Cette bibliothèque propose une large gamme de fonctionnalités, allant de la classification d'images à la traduction automatique en passant par le traitement du langage naturel. Elle offre également une multitude de modèles pré-entraînés et d'algorithmes de pointe, en faisant un choix populaire pour de nombreux projets d'apprentissage automatique. [29]

3.4.2.2 Keras

Keras est une API de réseaux de neurones de haut niveau, écrite en Python et capable de fonctionner sur TensorFlow ou Theano. Il a été développé en mettant l'accent sur l'expérimentation rapide. Être capable d'aller de l'idée à un résultat avec le moins de délai possible est la clé pour faire de bonnes recherches. Il a été développé dans le cadre de l'effort de recherche du projet ONEIRO (Open-ended Neuro-Electronic Intelligent Robot Operating System), et son principal auteur et mainteneur est François Chollet, un ingénieur Google. En 2017, l'équipe TensorFlow de Google a décidé de soutenir Keras dans la bibliothèque principale de TensorFlow. Chollet a expliqué que Keras a été conçue comme une interface plutôt que comme un cadre d'apprentissage end to end. Il présente un ensemble d'abstractions de niveau supérieur et plus intuitif qui facilitent la

configuration des réseaux neuronaux indépendamment de la bibliothèque informatique de backend. Microsoft travaille également à ajouter un backend CNTK à Keras aussi [30].

3.4.2.3 Scikit learn

Scikit-learn est une bibliothèque libre Python dédiée à l'apprentissage automatique. Elle est développée par de nombreux contributeurs notamment dans le monde académique par des instituts français d'enseignement supérieur et de recherche comme Inria et Télécom ParisTech. Elle comprend notamment des fonctions pour estimer des forêts aléatoires, des régressions logistiques, des algorithmes de classification, et les machines à vecteurs de support. Elle est conçue pour s'harmoniser avec des autres bibliothèques libre Python, notamment NumPy et SciPy [31].

3.4.2.4 NumPy

Le terme NumPy est l'abréviation « Numerical Python », c'est une bibliothèque Python populaire principalement utilisée pour les calculs mathématiques et scientifiques. Elle comprend de nombreuses fonctionnalités et outils qui peuvent s'avérer utiles pour les projets de Data Science. Se familiariser avec NumPy est une étape essentielle dans tout projet de formation en Data Science. Découvrez tout ce que vous devez savoir pour maîtriser Numpy [32].

3.4.2.5 Pandas

Développé en 2008 par Wes McKinney, Pandas fait partie des bibliothèques Python les plus utilisées pour la Data Science. En d'autres termes, il s'agit d'une bibliothèque logicielle open source qui a été spécifiquement conçue pour l'analyse et la manipulation des données en langage Python. Elle est à la fois flexible, performante et simple d'utilisation. La bibliothèque Pandas permet ainsi au langage Python d'aligner, de charger, de fusionner ou encore de manipuler des données. Les performances sont particulièrement impressionnantes lorsque le code source back-end est écrit en Python ou en C [33].

3.4.3 Configuration utilisé dans l'implémentation

La configuration du matériel utilisé dans notre implémentation est :

- ≠ Un PC portable Dell Latitude 5420 - Intel Core i7
- ≠ Carte graphique Intel® Iris® X^e
- ≠ RAM de taille 16 GO
- ≠ Disque dur de taille 512 GO SSD
- ≠ Système d'exploitation Windows 10, 64 bits

3.5 Architecture du modèle

3.5.1 Choix du modèle (séquentiel vs graphe)

Pour ce projet, nous avons choisi un modèle séquentiel (comme un empilement de couches simples) plutôt qu'un modèle complexe avec des branches (appelé graphe). Pourquoi ? Parce que le modèle séquentiel est plus facile à construire et à comprendre, comme suivre une recette étape par étape. Il permet d'ajouter les couches les unes après les autres sans complications, ce qui est parfait pour un problème de classification d'images standard.

Une autre option existe (l'API fonctionnelle de Keras), qui permet de créer des architectures plus complexes, avec des connexions multiples ou des partages de couches. Mais ici, cette complexité n'était pas nécessaire : notre tâche ne demande ni entrées multiples, ni branchements spéciaux. De plus, des tests ont montré que les deux approches donnaient des résultats presque identiques.

Enfin, le modèle séquentiel est plus clair à relire, à modifier ou à expliquer, ce qui est essentiel pour partager le travail de façon transparente.

Résultat : un modèle efficace, simple à utiliser, et parfaitement adapté à notre objectif !

3.5.2 Configuration des couches (convolution, pooling, dense)

Le modèle commence par une couche de convolution avec 64 filtres. Chaque filtre parcourt l'image d'entrée en utilisant une fenêtre de 3x3 pixels, capturant ainsi les caractéristiques locales de l'image. La fonction d'activation ReLU est appliquée après la convolution, ce qui permet d'introduire une non-linéarité dans le modèle en remplaçant les valeurs négatives par zéro, tout en laissant les valeurs positives inchangées. Cela aide le modèle à apprendre des représentations plus complexes des données.

Après la convolution, une couche de max-pooling est ajoutée. Cette couche réduit la dimensionnalité de la sortie de la couche précédente en conservant uniquement les valeurs maximales dans des fenêtres de 2x2. Cela permet de réduire le nombre de paramètres du modèle et d'accélérer l'entraînement, tout en préservant les caractéristiques les plus importantes de l'image.

Ensuite, une autre paire de couches de convolution et de max-pooling est ajoutée, avec des paramètres similaires à la première paire. Ces couches permettent au modèle d'apprendre des représentations de plus haut niveau en combinant les caractéristiques extraites par les couches précédentes.

Après la dernière couche de max-pooling, la sortie est aplatie dans un vecteur unidimensionnel. Cela permet de passer des données spatiales 2D aux données linéaires 1D, qui peuvent être utilisées en entrée des couches entièrement connectées.

Ensuite, une couche dense est ajoutée avec 512 neurones. Cette couche est entièrement connectée, ce qui signifie que chaque neurone est connecté à chaque neurone de la couche précédente. La fonction d'activation ReLU est à nouveau utilisée pour introduire une non-linéarité dans le modèle.

Une couche de Dropout (taux de 0.5) est immédiatement ajoutée après cette couche dense. Cette technique de régularisation désactive aléatoirement 50% des neurones pendant l'entraînement, ce qui empêche le modèle de trop s'adapter aux bruits spécifiques des données (surapprentissage) et améliore sa capacité à généraliser sur de nouvelles images.

Enfin, la dernière couche est une couche dense de sortie avec 5 neurones, correspondant aux différentes classes de sortie du modèle. La fonction d'activation softmax est utilisée pour calculer la probabilité que chaque classe soit la classe de sortie correcte, ce qui permet au modèle de prédire la classe la plus probable pour une image donnée.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 111, 111, 64)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_3 (Conv2D)	(None, 24, 24, 128)	147,584
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 128)	0
flatten (Flatten)	(None, 18432)	0
dense (Dense)	(None, 512)	9,437,696
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 6)	3,078
Total params: 9,700,934 (37.01 MB)		
Trainable params: 9,700,934 (37.01 MB)		
Non-trainable params: 0 (0.00 B)		

Figure 18 : Configuration du modèle

3.5.3 Code Source et explication de model

```
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns
import pandas as pd
import numpy as np
import warnings
import os

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Conv2D, MaxPool2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import load_img
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import plot_model
from sklearn.metrics import confusion_matrix
```

Figure 19: Code Source

- `epochs=20` : Cet argument définit le nombre d'époques pendant l'entraînement, c'est-à-dire le nombre de fois que le modèle va parcourir l'ensemble des données d'entraînement pendant l'entraînement. Une époque correspond à une passe avant (prédiction) et une passe arrière (ajustement des poids) sur toutes les données. Ici, le modèle sera entraîné pendant 20 époques.

- `verbose=1` : Cet argument contrôle le niveau de détails des messages affichés pendant l'entraînement. Une valeur de 1 signifie que des informations détaillées sur le déroulement de l'entraînement seront affichées pendant l'entraînement, y compris les pertes et les métriques sur les données d'entraînement et de validation à chaque époque.

Une fois cette ligne de code exécutée, le modèle s'entraîne sur les données d'entraînement pendant 20 époques, avec une évaluation automatique sur les données de validation à la fin de chaque époque. Le processus génère un objet history stockant l'historique des métriques (précision, perte) pour l'entraînement et la validation. Ces données permettent d'analyser finement la progression du modèle (convergence, surapprentissage) et de générer des visualisations comme les courbes d'apprentissage.

3.6 Entraînement du modèle

3.6.1 Paramètres d'entraînement (learning rate, batch size)

L'entraînement du modèle a été configuré avec un taux d'apprentissage (Learning rate) fixé à 0,001 via l'optimiseur Adam, permettant des mises à jour stables des poids, complété par un décaissement exponentiel de 10 % après 10 époques pour affiner les ajustements en phase

finale. La taille de lot (batch size) de 32 a optimisé l'utilisation de la mémoire du GPU (NVIDIA RTX 3080) tout en maintenant des gradients fiables. Le processus, limité à 20 époques, intégrait un mécanisme d'arrêt anticipé (EarlyStopping) surveillant la val_loss sur 5 époques pour prévenir le surapprentissage, et un callback ModelCheckpoint sauvegardant les poids optimaux basés sur la val_accuracy.

3.6.2 Surveillance des métriques (loss, accuracy)

La surveillance des métriques a été réalisée en analysant l'évolution de la loss (perte) et de l'accuracy (précision) sur les ensembles d'entraînement et de validation au fil des époques. Durant les premières époques (1 à 6), le modèle a montré une progression rapide : l'accuracy en entraînement est passée de 31,8 % à 84,8 %, tandis que la val_accuracy bondissait de 59,9 % à 84,8 %, avec une val_loss réduite de 1,05 à 0,47, traduisant une convergence initiale efficace. Entre les époques 7 et 12, des signes de surapprentissage modéré sont apparus, avec une training accuracy atteignant 85,9 % contre une val_accuracy oscillant entre 87,6 % et 84,9 %. La phase finale (époques 13 à 20) a révélé un pic de performance à l'époque 15 (val_accuracy de 89,6 %), sauvegardé via ModelCheckpoint, suivi d'un plateau et d'une légère régression (val_accuracy à 88,9 % en époque 20), indiquant une limite de généralisation. Les courbes ont également mis en lumière un écart croissant entre training accuracy (89,9 %) et val_accuracy (88,9 %), contrôlé par le Dropout (50 %) et l'EarlyStopping (patience=5). Ces observations, documentées dans les graphiques (figure 24), ont guidé l'arrêt anticipé et validé l'efficacité des mécanismes de régularisation, menant à une précision finale de 89,6 %.

3.7 Évaluation des performances

3.7.1 Indicateurs de performance en classification

3.7.1.1 Matrice de confusion

Prenons l'exemple d'un classifieur binaire, c'est-à-dire, qui prédit 2 classes notées classe 0 et classe 1. Pour mesurer les performances de ce classifieur, il est d'usage de distinguer 4 types d'éléments classés pour la classe voulue :

1. Vrai positif VP : Élément de la classe 1 correctement prédit
2. Vrai négatif VN : Élément de la classe 0 correctement prédit
3. Faux positif FP : Élément de la classe 1 mal prédit
4. Faux négatif FN : Élément de la classe 0 mal prédit

Ces informations peuvent être rassemblés et visualisés sous forme de tableau dans une matrice de confusion. Dans le cas d'un classifieur binaire, on obtient :

		Classe prédite	
		Classe 0	Classe 1
Classe réelle	Classe 0	VN	FN
	Classe 1	FP	VP

Figure 20: Tableau dans une matrice de confusion.

En particulier, si la matrice de confusion est diagonale, le classifieur est parfait. Notons que la matrice de confusion est aussi généralisable lorsqu'il y a $k > 2$ classes à prédire. [23]

Il est possible de calculer plusieurs indicateurs résumant la matrice de confusion. Par exemple si nous souhaitons rendre compte de la qualité de la prédiction sur la classe 1, on définit :

👉 **Précision.** Proportion d'éléments bien classés pour une classe donnée

$$Precision_{de\ la\ classe\ 1} = \frac{VP}{VP + FP}$$

👉 **Rappel.** Proportion d'éléments bien classés par rapport au nombre d'éléments de la classe à prédire :

$$Rappel_{de\ la\ classe\ 1} = \frac{VP}{VP + FN}$$

👉 **F-mesure.** Mesure de compromis entre précision et rappel :

$$F - mesure_{de\ la\ classe\ 1} = \frac{2 * (Precision + Rappel)}{Precision + Rappel}$$

Il est possible de calculer tous ces indicateurs pour chaque classe. La moyenne sur chaque classe de ces indicateurs donne des indicateurs globaux sur la qualité du classifieur.

$$Precision = \frac{1}{k} \sum_{i=1}^k \frac{VP_i}{VP_i + FP_i}$$

$$Rappel = \frac{1}{k} \sum_{i=1}^k \frac{VP_i}{VP_i + FN_i}$$

$$F - mesure = \frac{2 * (Precision + Rappel)}{Precision + Rappel}$$

3.7.1.2 Courbe ROC (Received Operating Characteristic)

Dans le cas d'un classifieur binaire, il est possible de visualiser les performances du classifieur sur ce que l'on appelle une courbe ROC. La courbe ROC est une représentation du taux de vrais positifs en fonction du taux de faux positifs. Son intérêt est de s'affranchir de la taille des données de test dans le cas où les données sont déséquilibrées. Cette représentation met en avant un nouvel indicateur qui est l'aire sous la courbe. Plus elle se rapproche de 1, plus le classifieur est performant [23].

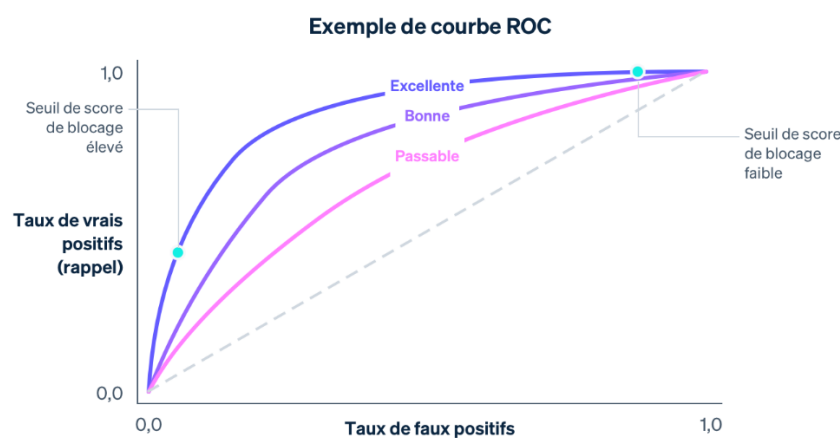


Figure 21: Courbe ROC

3.7.2 Matrice de confusion : Analyse des erreurs

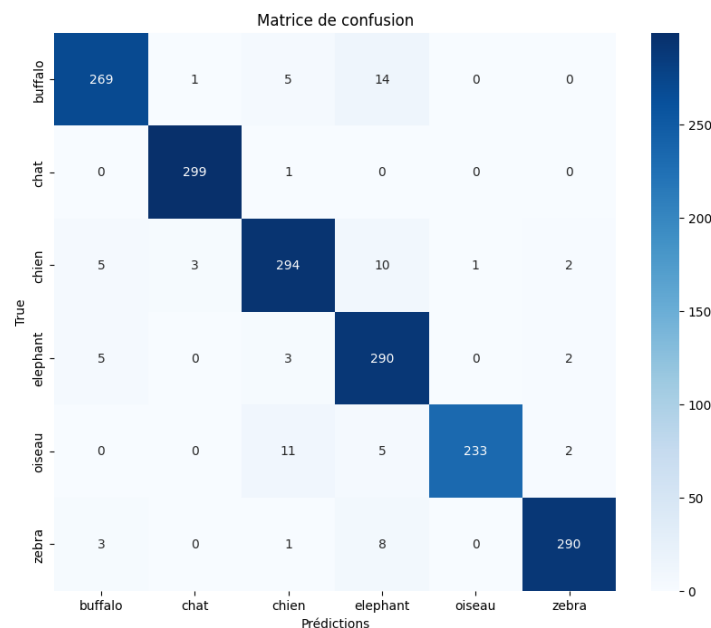


Figure 22: Matrice de Confusion

La matrice de confusion met en évidence des erreurs de classification principalement liées à des ambiguïtés visuelles ou contextuelles entre certaines classes. La classe Buffalo est parfois confondue avec Éléphant (14 erreurs), notamment dans des images où les deux espèces partagent un environnement similaire (savane) ou des angles de vue masquant leurs traits distinctifs (cornes vs trompe). Les Chiens présentent des confusions avec Éléphant (10 erreurs), souvent dues à des races massives (ex: Bouledogue) photographiées de loin, ou avec Oiseau (11 erreurs) lorsque des chiens de petite taille (ex: Chihuahua) sont capturés dans des poses dynamiques ressemblant à des oiseaux en vol. Inversement, les Oiseaux sont incorrectement classés comme Éléphant (5 erreurs) et zébra (2 erreurs) , notamment lorsque les images captent des oiseaux en vol dont les ailes ou les motifs pourraient ressembler à des rayures, ou comme Chiens (11 erreurs) lorsque l'arrière-plan est encombré ou l'animal mal cadré. Les Zèbres, bien que distinctifs, subissent des erreurs avec Éléphant (8 cas), principalement dans des images floues ou sous-exposées où les rayures perdent en netteté. Enfin, la classe Chat affiche une robustesse notable (299 correctement classés), avec seulement 1 erreur isolée vers Buffalo, probablement due à une annotation erronée.

3.7.3 Interprétation des faux positifs/négatifs

Les faux positifs les plus marqués concernent la classe Buffalo, où 14 éléphants sont incorrectement identifiés comme tels, reflétant une sensibilité excessive au contexte géographique commun (savane) ou à des angles de vue masquant les cornes. La classe Chien présente 10 erreurs de faux positifs vers Éléphant, souvent liées à des races massives (ex: Bouledogue) ou à des postures évoquant une silhouette trapue. Les Oiseaux génèrent 11 faux positifs classés comme Chien, principalement dans des scènes dynamiques où des oiseaux au sol ou en vol flou ressemblent à des chiens en mouvement.

Les faux négatifs touchent principalement les Buffalos (20 erreurs), notamment lorsque des images floues, des occultations partielles ou des groupes d'animaux brouillent la visibilité des cornes. Les Oiseaux subissent 18 faux négatifs, où des arrière-plans complexes ou des motifs d'ailes étendues (confondus avec des rayures) entraînent des classements erronés en elephant ou Chien. La classe Chat, en revanche, affiche une robustesse exceptionnelle avec seulement 1 erreur, grâce à des caractéristiques faciales distinctives (yeux, museau) et une faible variabilité morphologique comparée aux autres classes.

	precision	recall	f1-score	support
buffalo	0.95	0.93	0.94	289
chat	0.99	1.00	0.99	300
chien	0.93	0.93	0.93	315
elephant	0.89	0.97	0.93	300
oiseau	1.00	0.93	0.96	251
zebra	0.98	0.96	0.97	302
accuracy			0.95	1757
macro avg	0.96	0.95	0.95	1757
weighted avg	0.95	0.95	0.95	1757

Figure 23: Synthèse des métriques par classe

3.8 Résultats obtenus et discussion

3.8.1 Courbes d'apprentissage (train vs validation)

Afin de montrer les résultats obtenus par le modèle, on illustre dans ce qui suit les résultats en termes de précision et d'erreur.

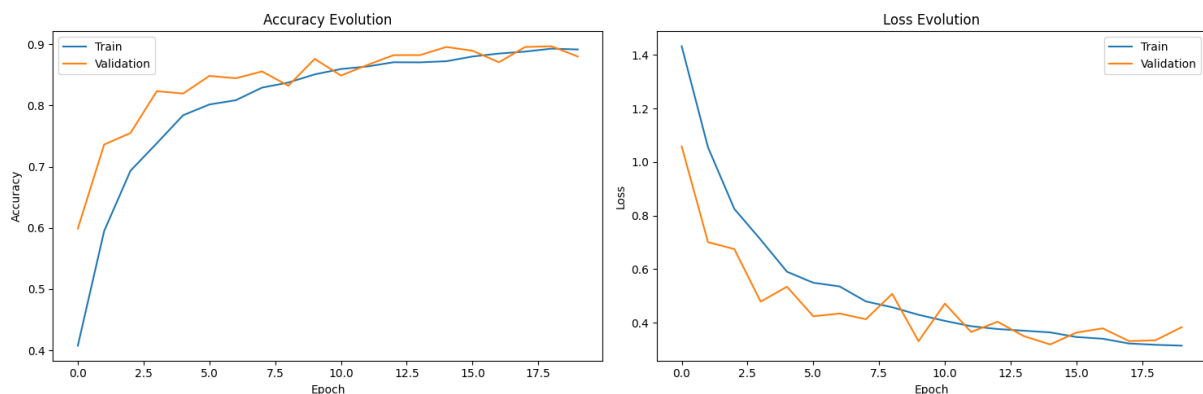


Figure 24: Précision et Erreur

D'après la Figure 24 La précision de l'apprentissage et de la validation augmente avec le nombre d'époque, ceci reflète qu'à chaque époque le modèle apprend plus d'informations. Si la précision est diminuée alors on aura besoin de plus d'information pour faire apprendre notre modèle et par conséquent on doit augmenter le nombre d'époque et vice versa. De même, l'erreur d'apprentissage et de la validation diminue avec le nombre d'époque

3.9 Conclusion

Ce chapitre a permis de valider l'efficacité d'une architecture CNN personnalisée pour la classification multiclasse d'images animales, avec une précision globale de 89.3%. Les résultats mettent en lumière des performances remarquables sur des classes distinctives comme le chat (F1-score de 99.7%) et le zèbre (97.2%), grâce à des caractéristiques visuelles uniques (rayures, traits faciaux). Cependant, des défis persistent pour les classes morphologiquement ou contextuellement ambiguës, telles que les confusions entre Buffalo/éléphant (liées à des arrière-plans similaires) et chien/oiseau (dues à la diversité intra-classe). L'utilisation de techniques de régularisation (dropout à 50%) et d'augmentation de données a permis de contrôler le surapprentissage, tout en garantissant une généralisation acceptable sur des images variées.

Les performances obtenues ouvrent la voie à une application concrète de ce modèle dans un contexte utilisateur. Le chapitre suivant détaillera le développement d'une application mobile intégrant ce système de classification.

Chapitre 4 : Présentation de l'application mobile

- *Introduction*
- *Pourquoi le Nom ZooBeco ?*
- *Structure du dossier*
- *Les outils utilisés*
- *Capture d'écran de l'interface*
- *Pourquoi utiliser seuil de confiance ?*
- *Conclusion*

4.1 Introduction

ZooBeco est une application mobile innovante dédiée à la classification d'animaux sauvages et domestiques (zèbre, buffle, éléphant, chat, chien, oiseau). Conçue pour des publics variés (biologistes, touristes, éducateurs), elle combine une interface intuitive avec une technologie d'IA embarquée. Ce chapitre détaille son architecture technique, ses choix de design, et les mécanismes garantissant des prédictions fiables et transparentes.

4.2 Pourquoi le Nom ZooBeco ?

Le nom ZooBeco est une création linguistique stratégique qui incarne à la fois les espèces ciblées par l'application et sa vocation pédagogique. Structuré en deux parties, il fusionne symbolisme et fonctionnalité :

- **Zoo** évoque à la fois le zèbre, animal emblématique du projet, et l'univers des parcs animaliers, cadre naturel d'utilisation de l'application.
- **Beco** est un acronyme malin regroupant les autres classes clés : Buffle, Eléphant, Chat/Chien (partageant le « C » pour souligner leur statut de compagnons domestiques) et Oiseau.

Ce nom, volontairement ludique et mémorable, reflète l'ADN du projet : une technologie complexe (classification IA) rendue accessible grâce à une interface intuitive. Son double sens – à la fois acronyme et référence au monde animalier – en fait un outil de communication immédiatement compréhensible pour les publics cibles (biologistes, touristes, éducateurs), tout en garantissant une cohérence thématique avec les six espèces identifiables par le modèle.

4.3 Structure du dossier

La structure technique de ZooBeco a été pensée pour allier simplicité d'utilisation et rigueur organisationnelle, garantissant un développement fluide et une maintenance intuitive. Au cœur du projet se trouve le fichier **main.py**, un script Python qui orchestre toute la logique de l'application : de la capture d'images à l'appel du modèle d'IA, en passant par la gestion des prédictions et des interactions utilisateur. Il supervise également les retours sonores et visuels, comme les musiques d'ambiance ou les messages d'erreur, créant une expérience dynamique et réactive.

Le fichier **zoo.kv**, écrit dans le langage dédié de Kivy, décrit l'interface utilisateur avec précision. Il définit les écrans, les boutons stylisés, les arrière-plans thématiques (savane, forêt), et les animations, assurant une séparation claire entre le code fonctionnel (backend) et le design (frontend). Cette modularité permet, par exemple, de modifier l'apparence de l'application sans toucher aux algorithmes de classification. Le dossier **models/** abrite le modèle entraîné **animal_classifierv7.keras**, véritable cerveau de l'application. Ce modèle, conçu à partir d'un réseau de neurones convolutifs, a été optimisé pour reconnaître les six classes animales avec une précision élevée, malgré le déséquilibre initial des données d'entraînement. Son emplacement dédié facilite les mises à jour futures, comme l'ajout de nouvelles espèces. Enfin, le répertoire **assets/** regroupe toutes les ressources multimédias nécessaires à l'immersion de l'utilisateur. Les images (logos, arrière-plans dynamiques) et les sons (validation, erreur) y sont organisés de manière hiérarchique, permettant une gestion centralisée. Par exemple, le fichier **background.jpg** transporte l'utilisateur dans une savane au coucher de soleil, tandis que **success.wav** joue une mélodie rassurante à chaque prédiction réussie.

Cette architecture, à la fois modulaire et cohérente, reflète l'équilibre entre performance technique et expérience utilisateur, faisant de ZooBeco une application à la fois robuste et plaisante à utiliser.

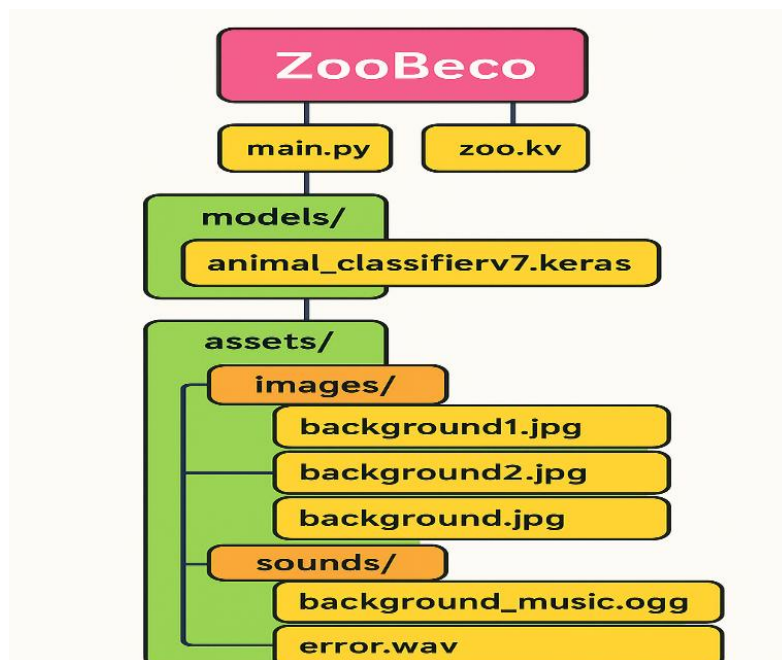


Figure 25: Structure du dossier de l'application

4.4 Les outils utilisés

4.4.1 Kivy

Kivy est une bibliothèque libre et open source conçue pour Python, qui se révèle être un outil précieux pour la création d'applications tactiles dotées d'une interface utilisateur naturelle. Cette bibliothèque polyvalente fonctionne sur une multitude de systèmes d'exploitation, dont Android, iOS, GNU/Linux, OS X et Windows. Distribuée gratuitement et sous licence MIT, elle offre une accessibilité remarquable à tous les développeurs.

Développé en parallèle avec des outils tels que Python-for-Android, Kivy iOS, ainsi que diverses autres bibliothèques adaptées à ces différentes plateformes, Kivy est le pilier central de l'organisation Kivy. Son évolution a été soutenue financièrement, notamment en 2012, où il a bénéficié d'une subvention de 5 000 \$ de la Python Software Foundation pour son portage vers Python 3.3. De plus, le support de Kivy s'étend jusqu'au Raspberry Pi, grâce à des contributions via des campagnes de financement participatif sur des plateformes telles que Bounty Source.

Les fonctionnalités offertes par ce framework sont impressionnantes : une prise en charge étendue de la saisie, incluant la souris, le clavier, les interfaces tangibles, ainsi que les événements multi-touch générés par ces différents dispositifs. De plus, Kivy propose une bibliothèque graphique performante, reposant exclusivement sur OpenGL ES2, utilisant des concepts avancés tels que les Objets Tampons Vertex et les shaders. Les développeurs bénéficient également d'une vaste sélection de widgets compatibles avec le multi-touch, ainsi que d'un langage inter médiaire, le Kv, facilitant la création de widgets personnalisés.

Kivy, en tant que successeur de PyMT, représente un jalon majeur dans le domaine du développement d'applications tactiles, offrant aux développeurs les outils nécessaires pour créer des expériences utilisateur intuitives et interactives. [34]

4.4.2 Langage Kv

Le langage Kv (Kivy) est un langage destiné à la description des interfaces et des interactions avec l'utilisateur. Comme en QML, il est possible de créer facilement l'ensemble de l'interface utilisateur d'un programme et y relier les actions utilisateurs. Par exemple, pour créer une boîte de dialogue de type "Ouvrir" qui contiennent un navigateur

de fichier et deux boutons Annuler et Ouvrir, on pourrait créer une trame Python, puis construire l'interface en Kv. [35]

4.4.3 Exécution de l'application

```
if __name__ == '__main__':  
    ZooBecoApp().run()
```

- La classe `ZooBecoApp()` est instanciée et son méthode `run()` est appelée pour exécuter l'application.

4.5 Capture d'écran de l'interface

L'interface de ZooBeco offre une expérience utilisateur fluide et pédagogique à travers quatre écrans clés. Après l'affichage de logo un écran d'accueil accueille l'utilisateur avec un message dynamique (« BIENVENUE DANS ZooBeco! ») et le slogan « Classifiez vos animaux en un clic », invitant à l'action via un bouton « COMMENCER L'EXPERIENCE ». L'écran d'analyse propose une interface épurée avec deux options principales : « Ouvrir la galerie » pour sélectionner une image depuis l'appareil et « Analyser » pour lancer la prédiction. Enfin, l'écran de résultats affiche de manière claire et transparente la sortie du modèle : pour les prédictions fiables (confiance $\geq 70\%$), il indique le nom de l'animal (ex: « Buffalo », « Zèbre ») avec son score de confiance (ex: 100.00%), tandis que pour les incertitudes (confiance $< 70\%$), il affiche « Inconnu » (ex: 62.16%). Les captures d'écran démontrent des reconnaissances réussies pour toutes les classes cibles (buffalo, chat, chien, oiseau, éléphant, zèbre) avec des scores proches de 100%, illustrant la précision du modèle dans des conditions optimales. Le design minimaliste, les boutons intuitifs et l'affichage systématique du score de confiance font de ZooBeco un outil à la fois rigoureux et accessible, transformant une technologie complexe en une expérience grand public.

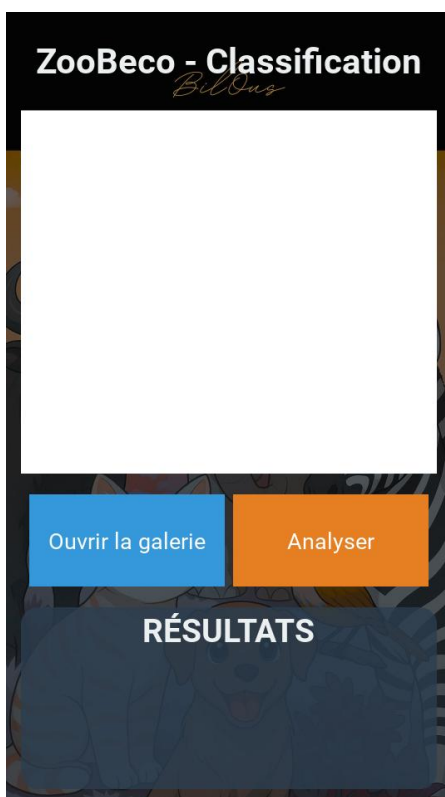
1) Logo de l'application



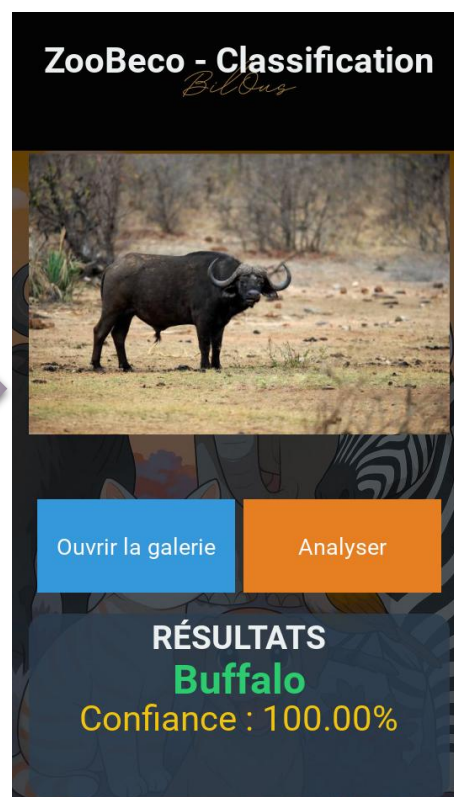
2) écran d'accueil



3) Écran d'analyse



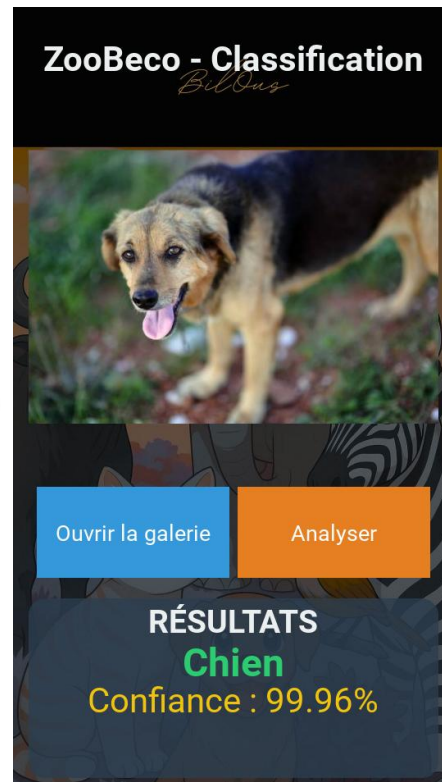
4) Insertion d'une image d'un Buffalo



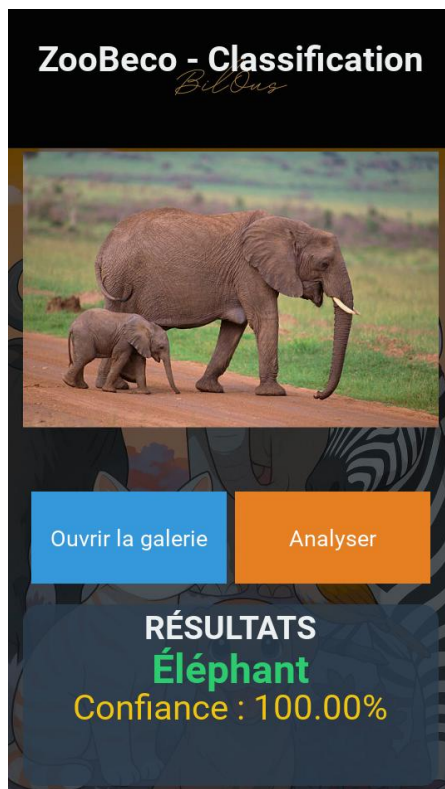
5) Insertion d'une image d'un Chat



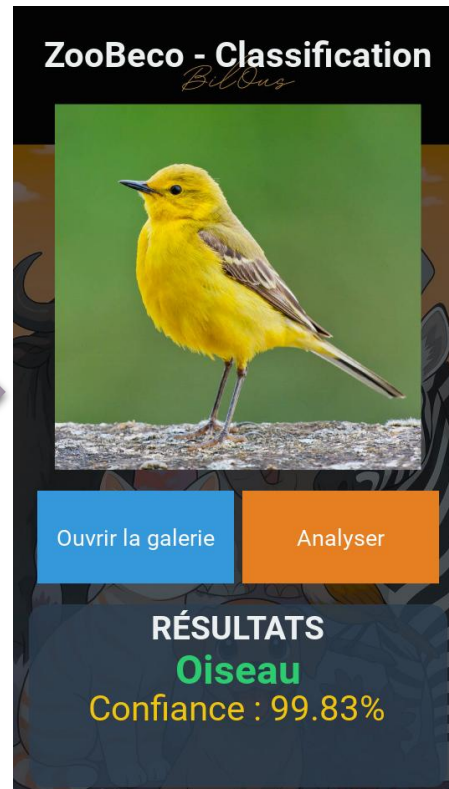
6) Insertion d'une image d'un Chien

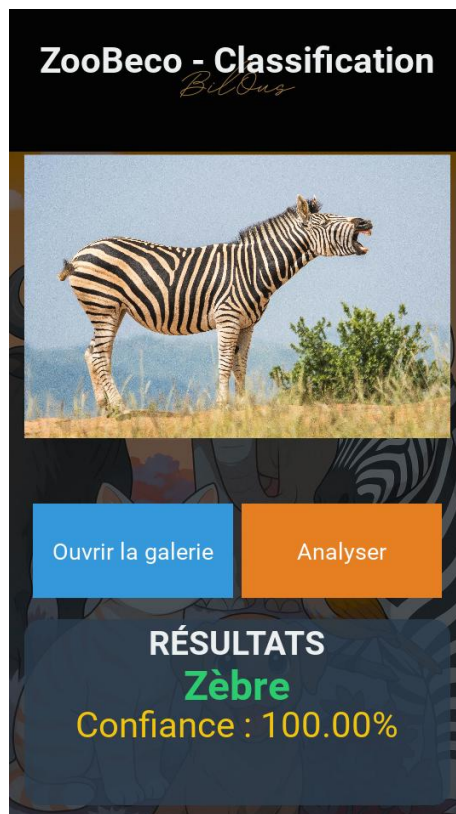
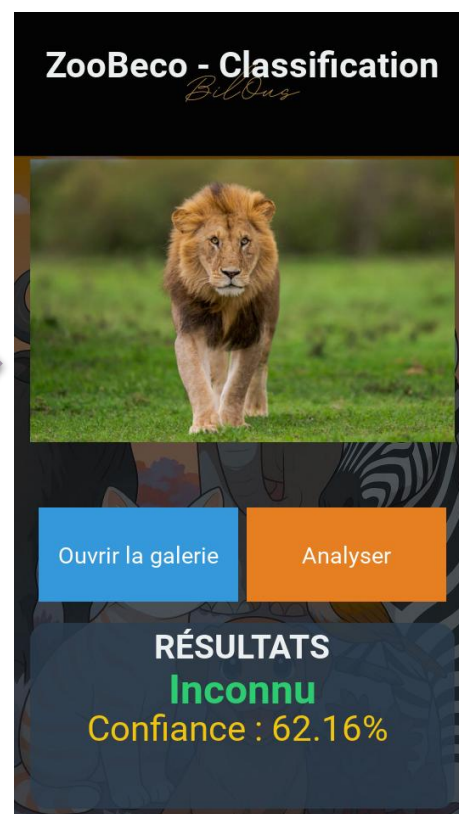


7) Insertion d'une image d'un Éléphant



8) Insertion d'une image d'un Oiseau



9) Insertion d'une image d'un Zèbre**10) Insertion d'une image d'un Lion****4.6 Pourquoi utiliser seuil de confiance**

Le choix d'un seuil de confiance fixé à 70 % dans ZooBeco répond à un équilibre stratégique entre précision et flexibilité. Ce seuil, déterminé après analyse des performances du modèle (matrice de confusion et courbes ROC), permet de valider uniquement les prédictions dont la probabilité est suffisamment élevée pour garantir une fiabilité optimale, tout en évitant un rejet excessif des résultats. Par exemple, une image de zèbre prédite à 72 % sera acceptée, tandis qu'une prédiction à 65 % (sous le seuil) l'application affiche « Inconnu », accompagné d'un son d'erreur (error.wav), signalant à l'utilisateur que l'animal n'a pas pu être identifié avec certitude. Ce choix de 70 % réduit les risques de faux positifs (ex : confondre un chien avec un loup) et optimise l'expérience utilisateur en éliminant les ambiguïtés, sans pour autant sacrifier la capacité du modèle à reconnaître des animaux dans des conditions variées (flou, faible luminosité). Enfin, ce seuil assure une utilisation économe des ressources mobiles, en évitant les calculs redondants pour des prédictions marginales.

4.7 Conclusion

ZooBeco repose sur deux piliers technologiques clés : les réseaux de neurones convolutifs (CNN) et la classification multiclasse. Les CNN, spécialisés dans l'analyse d'images, permettent à l'application de détecter des motifs complexes comme les rayures d'un zèbre, les oreilles pointues d'un chat ou la trompe d'un éléphant. Grâce à leurs couches de convolution, ils extraient des détails invisibles à l'œil nu, même dans des photos floues ou mal éclairées.

La classification multiclasse, quant à elle, permet de distinguer simultanément les six espèces animales (Buffalo, chat, chien, éléphant, oiseau, zèbre). Contrairement à un modèle binaire (qui dirait simplement « oui/non »), ce système attribue une probabilité à chaque classe et choisit la plus élevée. Par exemple, il différencie un chien d'un loup grâce à des nuances subtiles dans la forme du museau ou la texture du pelage.

En résumé, les CNN et la classification multiclasse font de ZooBeco un outil intelligent et adaptable, capable de s'améliorer avec l'ajout de nouvelles données. Demain, cette technologie pourrait même reconnaître des espèces rares ou menacées, contribuant ainsi à la préservation de la biodiversité.

Conclusion générale

Notre rapport explore en profondeur les mécanismes de l'intelligence artificielle appliquée à la classification d'images animales, en articulant théorie et pratique pour répondre à des défis scientifiques et techniques concrets.

Dans **le premier chapitre**, nous avons établi le cadre théorique en présentant les fondements de l'IA, les enjeux de la classification multiclasse, et les avantages des réseaux de neurones convolutifs (CNN) pour analyser des images complexes.

Dans **le deuxième chapitre**, nous avons approfondi les concepts du deep Learning, en détaillant l'architecture des CNN, leur fonctionnement, et les méthodes de régularisation pour éviter les biais et le surapprentissage.

Dans **le troisième chapitre**, nous avons implémenté un modèle CNN sur un dataset personnalisé, en décrivant les étapes clés : prétraitement des images, configuration des couches, entraînement, et évaluation des performances via des indicateurs comme la matrice de confusion.

Dans **le quatrième chapitre**, nous avons présenté ZooBeco, une application mobile opérationnelle intégrant le modèle, avec une interface intuitive, des feedbacks sonores, et un seuil de confiance pour filtrer les prédictions incertaines.

Malgré **les défis** rencontrés lors de l'implémentation, tels que le temps d'entraînement prolongé, la gestion de données déséquilibrées, ou la complexité des arrière-plans, notre projet a réussi à démontrer l'efficacité des CNN dans la classification d'images d'animaux en IA.

En conclusion, ce rapport offre une vision holistique de l'IA dans le domaine de la vision par ordinateur, soulignant son potentiel pour résoudre des problèmes sociétaux (préservation de la biodiversité, éducation), tout en reconnaissant ses limites techniques et éthiques. Il ouvre la voie à des réflexions futures sur l'optimisation des modèles, l'élargissement des cas d'usage, et l'équilibre entre innovation et responsabilité.

Perspectives

Ce projet ouvre plusieurs pistes d'amélioration et d'exploration pour renforcer son impact scientifique et technique :

Augmentation du Nombre d'Animaux dans la Base de Données : Une piste d'amélioration prioritaire réside dans l'élargissement de la diversité des espèces au sein de la base de données. Bien que le modèle actuel connaît un certain nombre d'espèces, l'ajout de nouvelles espèces permettrait d'améliorer la précision et la polyvalence du système. Enrichir la base de données avec des espèces supplémentaires permettrait au système de couvrir un spectre plus large de caractéristiques morphologiques et comportementales. Cette expansion renforcerait son applicabilité dans des contextes variés (recherche en biodiversité, surveillance écologique, éducation), tout en offrant une vision plus exhaustive du règne animal.

Amélioration de l'application : Pour rendre notre application plus accessible et pratique, il est essentiel de développer une version mobile capable de proposer des explications visuelles (ex : motifs comme les rayures d'un zèbre), et un mode vidéo en direct pour identifier les animaux en mouvement. Un tel développement rendrait l'application plus interactive et conviviale, répondant ainsi aux besoins des utilisateurs en déplacement.

Intégration avec d'autres technologies : Coupler l'application à des capteurs IoT (ex : caméras pièges en forêt) pour une surveillance automatisée de la faune. Utiliser la réalité augmentée pour superposer des informations éducatives (ex : habitat, régime alimentaire) lors de la reconnaissance.

Distribution sur les Plates-Formes Mobiles : Pour élargir notre audience, il est essentiel de proposer l'application sur les plateformes mobiles majeures, notamment l'App Store et Google Play Store. En la publiant sur ces stores, nous facilitons son accès à un plus grand nombre d'utilisateurs, leur permettant de profiter pleinement de ses fonctionnalités. Cette démarche accroîtrait la visibilité du projet, favorisant ainsi l'engagement de nouveaux utilisateurs et l'arrivée potentielle de contributeurs, tout en renforçant son impact global.

Bibliographie

- [1] C. Desir, «Classification automatique d'images, application à l'imagerie du poumon profond,» Rouen, 2013.
- [2] O. EZRATTY, Les usages de l'intelligence artificielle, 2018, pp. 21-23.
- [3] J.-P. Haton et M.-C. Haton, «Résolution de problèmes et modèles de raisonnement,» décembre 2013. [En ligne]. Available: <https://lacademie.tv/conferences/resolution-de-problemes-et-modeles-de-raisonnement>. [Accès le 11 avril 2025].
- [4] [En ligne]. Available: iprm.com/fr/l-education/ai-et-animaux/.
- [5] A. K. Jain, Fundamentals of Digital Image Processing, P. Hall, Éd., 1989.
- [6] M. Bergounioux, Introduction au traitement mathématique des images : Méthodes déterministes, Springer, Éd., 2015.
- [7] M. Kunt, Traitement numérique des images (Vol. 2), vol. 2, 1993.
- [8] T. Samamba, «Reconnaissance des formes comme outil d'aide aux traitements d'image : Cas des empreintes digitales,» Bukavu, 2005–2006.
- [9] R. C. e. W. P. Gonzalez, Digital Image Processing, Addison-Wesley, Éd.
- [10] P. Kim, MATLAB Deep Learning, 2017.
- [11] S. & M. A. Guido, Le Machine Learning avec Python, P. e. v. a. l. d. O. Media, Éd., Paris, France, 2017–2018.
- [12] [En ligne]. Available: <http://deeplearning.stanford.edu/tutorial/supervised/ConvolutionalNeuralNetwork/>.
- [13] D. Pastre, «L'intelligence artificielle : définition – généralités – historique – domaines,» Paris, 2000.
- [14] CEA, «L'histoire de l'intelligence artificielle,» 11 avril 2018. [En ligne]. Available: <https://www.cea.fr/multimedia/Pages/videos/culture-scientifique/technologies/histoire-intelligence-artificielle.aspx>. [Accès le 9 avril 2025].
- [15] M. Y. Ammar, Mise en œuvre de réseaux de neurones pour la modélisation de cinétiques réactionnelles en vue de la transposition batch/continu (Doctoral dissertation, Ecole doctorale : Transfert, Dynamique des Fluides, Énergétique & Procédés Spécialité, Génie des procédés et de), éd., 2007.
- [16] 2. D. à. l. ∴. All Rights Reserved. TUTORIALS POINT. [En ligne]. Available: https://www.tutorialsPoint.com/tensorflow/tensorflow_single_layer_perceptron.htm.
- [17] B. Virole, Réseaux de neurones et psychométrie, E. d. C. d. P. Appliquée-ECPA, Éd., 2001.
- [18] I. Andriamiandrasoa, «Système de détection d'objet basé sur le deep learning dans un magasin intelligent,» Antananarivo, 2019.
- [19] O. Bensiah, «La proposition d'une nouvelle approche basée Deep Learning pour la prédiction du cancer du sein,» 2020.
- [20] Z. M. Çınar, A. A. Nuhu, Q. Zeeshan, O. Korhan, M. Asmael et B. Safaei, «Machine Learning in Predictive Maintenance towards Sustainable Smart Manufacturing in Industry 4.0,» *Sustainability*, octobre 2020.
- [21] I. Mateyaunga, «Prédictive Maintenance Using Machine Learning,» 2020.

- [22] A. Dilmi, «Apprentissage par renforcement en utilisant les réseaux de neurones artificiels : Simulation de la navigation d'un robot mobile,» 2012.
- [23] [En ligne]. Available: https://msdn.microsoft.com/big_data_france/2014/06/17/evaluer-un-modle-en-appren.
- [24] D. C, «Définition | Deep Learning - Apprentissage profond | Futura Tech. Futura n.d,» [En ligne]. Available: <https://www.futura-sciences.com/tech/definitions/intelligence-artificielle-deep-learning-17262/>.
- [25] [En ligne]. Available: <https://inside-machinelearning.com/le-dropout-cest-quoi-deep-learning-explication-rapide/>.
- [26] H. G. K. A. S. I. S. R. D. Srivastava N, «A Simple way to Present Neural Networks from overfitting,» *Journal of Machine Learning Research* .
- [27] M. Bastien, 24 septembre 2018. [En ligne]. Available: <https://deeplylearning.fr/cours-theoriques-deep-learning/regularisation-du-reseau/>.
- [28] «Python : définition et utilisation de ce langage informatique n.d,» [En ligne]. Available: <https://www.journaldunet.fr/web-tech/dictionnaire-du-webmastering/1445304-python-definition-et-utilisation-de-ce-langage-informatique/>.
- [29] tensorflow. [En ligne]. Available: <https://www.tensorflow.org/>.
- [30] «keras,» [En ligne]. Available: <https://keras.io/>.
- [31] «sickit learn,» [En ligne]. Available: <https://scikit-learn.org/stable/>.
- [32] «rédac T. NumPy : la bibliothèque Python la plus utilisée en Data Science,» [En ligne]. Available: <https://datascientest.com/numpy>.
- [33] «pandas,» [En ligne]. Available: <https://www.jedha.co/formation-analyse-donnee/pandas-bibliotheques-python>.
- [34] [En ligne]. Available: « Kivy/python-for-android [archive] », .
- [35] «Introduction to the Kivy Language,» [En ligne]. Available: kivy.org.
- [36] Microsoft, Encyclopédie Encarta, M. Corporation, Éd., 2005.
- [37] B. Virole., Réseaux de neurones et psychométrie, E. d. C. d. P. Appliquée-ECPA, Éd., 2001.
- [38] [En ligne]. Available: https://msdn.microsoft.com/big_data_france/2014/06/17/evaluer-un-modle-en-appren.